

Introduction to AI Evasion Attacks

Evasion attacks manipulate inference-time inputs to cause a trained model to produce incorrect outputs. Adversarial machine learning, the discipline that studies these hostile interactions with machine learning systems, treats this interference during the inference phase as a distinct threat because it bypasses safeguards built into the training pipeline.

Understanding evasion is therefore essential for anyone securing deployed AI models, such as AI Red Teamers.

The AI Attack Landscape

To understand evasion, place it within the lifecycle of adversarial machine learning. AI systems can be attacked during training or at inference, and the distinction drives very different defenses.

Training-time attacks change what the model learns. Data poisoning injects or alters training samples so the model internalizes biased patterns. Label manipulation tampers with annotations so ground truth no longer matches reality. Trojan attacks implant hidden triggers that activate specific behavior when an attacker's pattern appears in the input.

These training-time attacks differ from inference-time manipulation, which changes only what the model sees when making a prediction. Data poisoning requires access to the dataset or pipeline and tends to shift behavior globally. Evasion, by contrast, leaves training and parameters untouched and succeeds by sending crafted inputs through the normal interface so a single example crosses the model's learned decision boundary.

A property called transferability links these settings to real deployments. An adversarial example crafted against a surrogate model can often fool different production models, particularly when architectures or data are similar. This enables offline preparation and black-box attacks where the defender exposes only an API but not internals.

Evasion Attacks in Traditional ML vs. LLMs

Evasion in traditional machine learning targets fixed feature representations, so small, controlled edits can push an example across a learned decision boundary. In a spam filter that uses a bag-of-words model, adding benign tokens changes term frequencies and therefore the posterior used for classification, while the message still reads the same to a human. In a static malware detector, rearranging sections or perturbing imports alters byte patterns and crafted signatures without changing program intent. The common thread is direct manipulation of the summarized features the model consumes, with the goal of moving a single prediction to the other side of its threshold at inference time.

By contrast, large language models produce open-ended text and follow instructions, which makes `prompt injection` the central evasion pattern. An attacker places adversarial directives into the prompt or surrounding context so the model prioritizes those instructions over the original `system` guidance, all without changing parameters. The effect is still inference-time only, yet the consequences differ because the output is itself an action surface, for example a block of code, a database query, or markup that another system might execute or render. When tool routing or API calling wraps the model, those outputs can trigger follow-on behavior that extends the attack beyond the initial response.

Both settings share the same core idea, modify only the input seen at prediction time and steer the model's behavior. The main difference lies in where the leverage comes from. Traditional models expose structured features and a label threshold, so evasion edits target the statistics that feed the classifier. LLMs expose conversational state and instruction following, so evasion edits target the model's decision process through carefully placed text and context management. This distinction explains why we begin with structured classifiers, the mechanics are easy to observe and measure, then we map the intuition to LLM prompts where the surface is richer but the objective is the same.

This Module's Focus

This module anchors these concepts through a hands-on technique: the `GoodWords` attack. This attack manipulates probabilistic spam filters by inserting carefully selected benign tokens. Because Naive Bayes models assume conditional independence between features, adding those tokens shifts posterior probabilities enough to force a misclassification without raising obvious suspicion.

Studying this method builds intuition that spans domains. The attack exploits model assumptions, highlights the difference between white-box and black-box knowledge, and underscores the trade-off between subtle perturbations and reliable success, which also generalize well to other classification tasks and highlight a fundamental approach to adversarial manipulation.

The GoodWords Attack

The GoodWords attack, first introduced by Lowd and Meek in their 2005 paper "[Good Word Attacks on Statistical Spam Filters](#)," is an adversarial technique designed to exploit fundamental probabilistic assumptions within `Naive Bayes` classifiers, particularly those deployed for spam detection. This attack methodology manipulates the classifier's decision-making process by strategically appending carefully selected legitimate words to malicious messages, disguising spam content with legitimate-looking terms.

At its core, the attack leverages the mathematical foundation of `Naive Bayes` classification, specifically targeting the classifier's reliance on word frequency distributions and conditional probabilities. The technique demonstrates how probabilistic models can be deceived through

simple manipulations that exploit their inherent assumptions about feature independence and probability calculations.

The GoodWords attack is simple and effective. Rather than attempting to obfuscate or modify the malicious content itself, the attack preserves the original spam message intact while augmenting it with additional tokens that shift the overall probability distribution. This approach maintains the semantic meaning and intent of the spam message while simultaneously convincing the classifier that the message belongs to the legitimate class.

Theoretical Foundation: Naive Bayes Classification Refresher

As we explored in the Fundamentals of AI module and saw applied in the Applications of AI in InfoSec module, Naive Bayes classifiers use Bayes' theorem to calculate the probability of a message belonging to a particular class. Here we'll briefly revisit the key concepts essential for understanding how the GoodWords attack exploits this algorithm.

Remember that Naive Bayes applies Bayes' theorem to classification:

$$P(C|D) = \frac{P(D|C) \cdot P(C)}{P(D)}$$

In spam detection, this calculates the probability that a message belongs to class

CC

(spam or ham) given document

DD.

As we covered earlier, the classifier's decision rule compares posterior probabilities:

$$\text{class} = \underset{c \in \{\text{spam}, \text{ham}\}}{\text{argmax}} P(c|D)$$

The critical "naive" assumption that makes this algorithm both efficient and vulnerable is the conditional independence of features. This allows the likelihood to factorize as:

$$P(D|C) = \prod_{i=1}^n P(w_i|C)$$

where each word

w_i

contributes independently to the probability calculation. To avoid numerical underflow, implementations work in log space:

$$\text{class} = \underset{c \in \{\text{spam}, \text{ham}\}}{\text{argmax}} [\log P(c) + \sum_{i=1}^n \log P(w_i|c)]$$

This independence assumption, while making Naive Bayes computationally efficient, creates a vulnerability. The additive nature of log probabilities means that classification decisions can be influenced by simply adding more words to a message. With this

understanding of the underlying probabilistic framework, we can now examine the specific mathematical mechanics that make the GoodWords attack so effective.

Attack Mechanics and Probability Manipulation

Think of the classifier as a scale weighing evidence for spam versus ham. Each word adds weight to one side or the other. Spam words like `FREE` and `WINNER` pile evidence onto the spam side. The attack works by adding enough legitimate words to the ham side that the scale tips, even though the original spam content remains unchanged. The beauty lies in how these weights combine through simple addition in log space, making the attack both predictable and powerful.

The attack's effectiveness comes from its manipulation of the probability calculations within the `Naive Bayes` algorithm.

When we consider a spam message

M_{spam}

with words

$\{w_1, w_2, \dots, w_m\}$

the classifier normally calculates:

$$\log P(\text{spam} | M_{\text{spam}}) = \log P(\text{spam}) + \sum_{i=1}^m \log P(w_i | \text{spam}) \quad \log P(\text{spam} | M_{\text{spam}}) = \log P(\text{spam}) + \sum_{i=1}^m \log P(w_i | \text{spam})$$

$$\log P(\text{ham} | M_{\text{spam}}) = \log P(\text{ham}) + \sum_{i=1}^m \log P(w_i | \text{ham}) \quad \log P(\text{ham} | M_{\text{spam}}) = \log P(\text{ham}) + \sum_{i=1}^m \log P(w_i | \text{ham})$$

These equations show how the classifier builds its decision. Start with the base probability that any message is spam or ham, then add up the evidence from each word. In plain terms, if a message contains `FREE`, `WINNER`, and `CLAIM`, each word contributes spam evidence that gets summed together. For successful classification as spam, we need:

$$\log P(\text{spam} | M_{\text{spam}}) > \log P(\text{ham} | M_{\text{spam}}) \quad \log P(\text{spam} | M_{\text{spam}}) > \log P(\text{ham} | M_{\text{spam}})$$

Now here's where the attack comes in. We take that same spam message and append carefully chosen legitimate words.

The attack modifies the message by appending a set of "good words"

$G = \{g_1, g_2, \dots, g_k\}$

creating an augmented message

$M_{\text{augmented}} = M_{\text{spam}} \cup G$

The new probability calculations become:

$$\log P(\text{spam} | M_{\text{augmented}}) = \log P(\text{spam}) + \sum_{i=1}^m \log P(w_i | \text{spam}) + \sum_{j=1}^k \log P(g_j | \text{spam}) \quad \log P(\text{spam} | M_{\text{augmented}}) = \log P(\text{spam}) + \sum_{i=1}^m \log P(w_i | \text{spam}) + \sum_{j=1}^k \log P(g_j | \text{spam})$$

$$\log P(\text{ham} | M_{\text{augmented}}) = \log P(\text{ham}) + \sum_{i=1}^m \log P(w_i | \text{ham}) + \sum_{j=1}^k \log P(g_j | \text{ham})$$

$$P(\text{ham} | M_{\text{augmented}}) = \log P(\text{ham}) + \sum_{i=1}^m \log P(w_i | \text{ham}) + \sum_{j=1}^k \log P(g_j | \text{ham})$$

Notice the new terms at the end of each equation. The original spam words are still there, contributing the same evidence they always did. But now we've added good words that strongly favor ham. Each good word adds more weight to the ham side of the scale than to the spam side. Add enough of them, and the scale tips. The attack succeeds when the addition of good words reverses the inequality, causing:

$$\log P(\text{ham} | M_{\text{augmented}}) > \log P(\text{spam} | M_{\text{augmented}}) \quad \log P(\text{ham} | M_{\text{augmented}}) > \log P(\text{spam} | M_{\text{augmented}})$$

When does this reversal happen? We need the good words to contribute more ham evidence than the original message contributed spam evidence. Mathematically, this reversal occurs when:

$$\sum_{j=1}^k [\log P(g_j | \text{ham}) - \log P(g_j | \text{spam})] > \sum_{i=1}^m [\log P(w_i | \text{spam}) - \log P(w_i | \text{ham})] + [\log P(\text{spam}) - \log P(\text{ham})]$$

$$\sum_{j=1}^k [\log P(g_j | \text{ham}) - \log P(g_j | \text{spam})] > \sum_{i=1}^m [\log P(w_i | \text{spam}) - \log P(w_i | \text{ham})] + [\log P(\text{spam}) - \log P(\text{ham})]$$

Breaking this down, the left side represents the net contribution of our good words. Each good word shifts the balance toward ham by the difference between its ham probability and spam probability. If `meeting` appears frequently in legitimate messages but rarely in spam, it contributes strongly to the left side. The right side represents the hurdle we need to overcome: the original spam signal plus any inherent bias the classifier has toward one class or another.

For example, imagine a message with strong spam words contributing a combined difference of 8.0 toward spam. If we add words like `meeting`, `tomorrow`, and `thanks` that each contribute 3.0 toward ham, we need at least three such words ($3 \times 3.0 = 9.0$) to overcome the spam signal and tip the classification.

The attack's goal is to select words

`gj`

that maximize the difference

$\log P(g_j | \text{ham}) - \log P(g_j | \text{spam})$
 $\log P(g_j | \text{ham}) - \log P(g_j | \text{spam})$,
effectively overwhelming the spam signal with legitimate-looking features.

Good Word Selection Strategy

The effectiveness of the GoodWords attack depends on selecting appropriate words to append. The optimal selection strategy identifies words with strong discriminative power in favor of the legitimate class. We define a "goodness score" for each word

ww

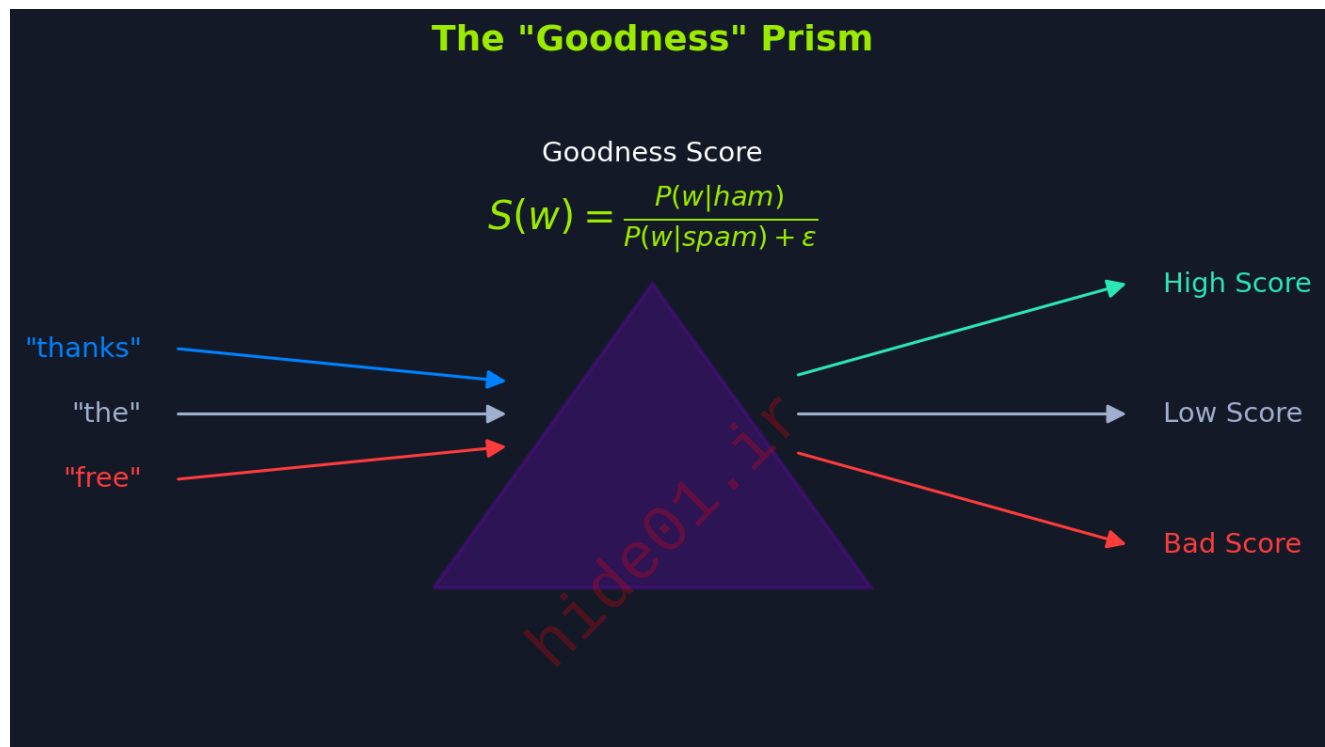
as:

$$S(w) = \frac{P(w|ham)}{P(w|spam) + \epsilon}$$

where

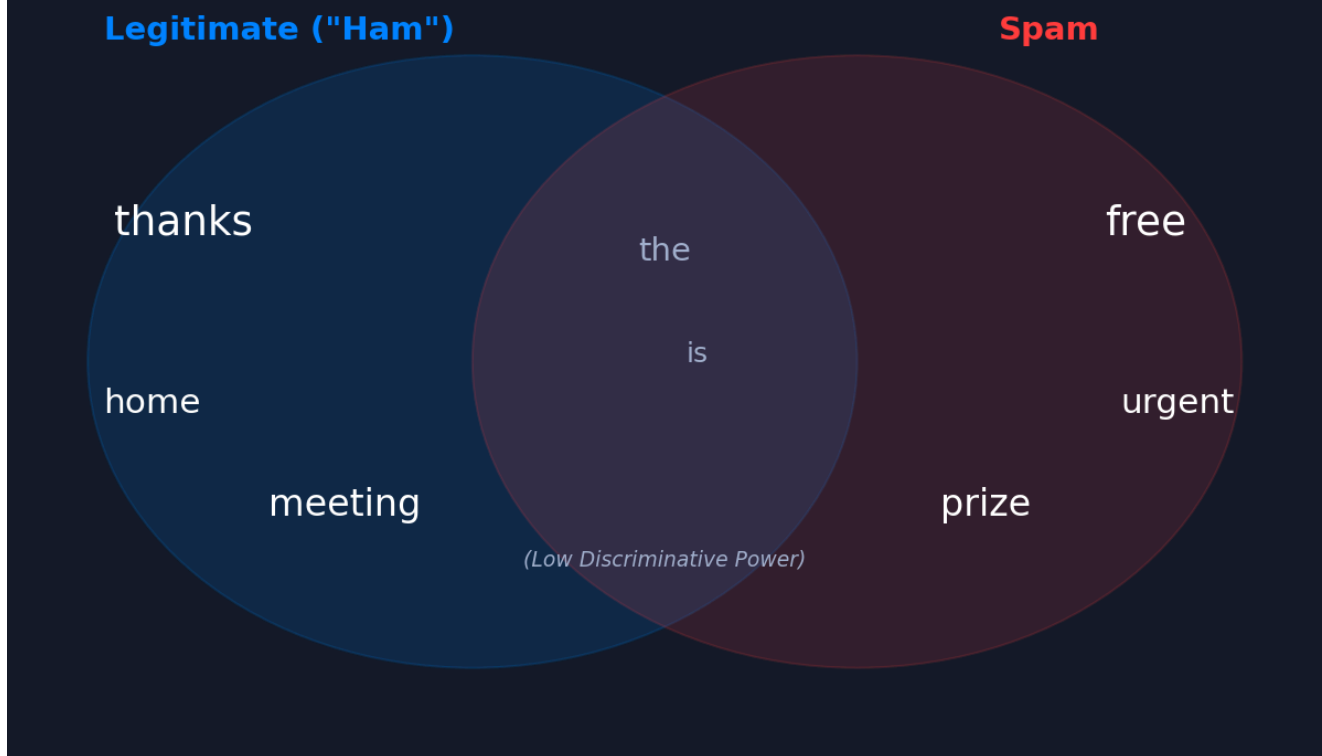
ϵ is a small constant to prevent division by zero.

Words with high goodness scores appear frequently in legitimate messages while being rare or absent in spam messages. Consider an example: if the word "meeting" appears in 45



We employ a systematic approach to word selection in our attack implementation. First, we analyze the training corpus to compute conditional probabilities for all vocabulary terms. We then rank words by their goodness scores, creating a prioritized list of candidates. This ranking considers not only the raw probability ratios but also the absolute frequencies to ensure selected words are representative of legitimate communication patterns.

The Two Vocabulary Worlds



When selecting words, we must avoid those that might trigger other spam detection heuristics. While a word might have a high goodness score, its inclusion could activate rule-based filters or appear suspicious to human reviewers. The selection process often incorporates additional constraints to ensure the augmented message maintains linguistic coherence and plausibility.

The number of words to add represents an important parameter balancing attack effectiveness against message length constraints. Adding too few words may fail to shift the probability sufficiently, while adding too many could make the message appear unnatural or exceed transmission limits. Analysis shows that most spam messages can be successfully misclassified by adding between 15 and 30 carefully selected good words, achieving evasion rates over 90%.

Why the Attack Works: Exploiting Independence Assumptions

The defining assumption of Naive Bayes - conditional independence of words given the class - is both its strength and weakness. This simplification is computationally efficient and often effective in practice, yet it also explains why the GoodWords attack works. Because the likelihood factorizes as:

$$P(D|C) = \prod_{i=1}^n P(w_i|C) \quad P(D \mid C) = \prod_{i=1}^n P(w_i \mid C)$$

Each word contributes independently to the total score. In $\log$ space the contributions add rather than multiply, so additional words simply shift the running sum. An attacker can

therefore append words that have strongly skewed conditional probabilities toward the legitimate class, incrementally pushing the decision toward `ham` even when the original content is characteristic of `spam`.

The classifier does not model word relationships or broader context, so it cannot penalize the semantic mismatch between the appended legitimate words and the underlying spammy message. Given enough favorable additions, quantity overwhelms quality, and the accumulated evidence overturns the initial spam signal.

Several implementation choices increase this susceptibility. Training typically fixes word-class probabilities from historical data, producing relatively static distributions that adversaries can study. Once those distributions are understood, the most advantageous `GoodWords` for augmentation can be identified systematically. Standard smoothing methods such as `Laplace smoothing` assign non-zero probability to unseen or rare words, which prevents rejection of unusual combinations and increases the range of words available to an attacker. Together, the independence assumption, the additive scoring in `log space`, the static learned distributions, and the effects of smoothing create the conditions that the `GoodWords` attack exploits.

Spam Filter Implementation

To demonstrate the `GoodWords` attack, we need a target to attack. To this end, we will build a `Naive Bayes` spam classifier trained on real SMS data.

Initial Setup and Dependencies

First, we'll import the necessary libraries and set seeds for reproducibility. We'll organize our imports into three groups, standard library, third-party packages, and our custom HTB utilities:

```
import json
import pickle
import random
import re
import urllib.request
import zipfile
from pathlib import Path
import numpy as np

# Reproducibility
random.seed(1337)
np.random.seed(1337)
```

Next, we'll import the data science and machine learning libraries used to build the classifier:


```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, accuracy_score
```

We'll also import the HTB library utilities for consistent styling and helper functions:

```
from htb_ai_library import (
    AZURE,
    HACKER_GREY,
    HTB_GREEN,
    MALWARE_RED,
    NODE_BLACK,
    NUGGET_YELLOW,
    WHITE,
    AQUAMARINE,
    load_model,
    save_model,
)
```

The seed value `1337` ensures our results remain reproducible across runs, which is essential for verifying attack effectiveness consistently. The HTB library provides both functionality for model persistence and consistent color constants that follow the HTB visual theme. Using `pathlib.Path` instead of string concatenation handles Windows and Linux path differences automatically. For example, `Path("data") / "file.txt"` produces `data\file.txt` on Windows and `data/file.txt` on Linux.

Loading the SMS Spam Dataset

We'll use the SMS Spam Collection dataset from the UCI Machine Learning Repository, which contains 5,574 labeled messages. To avoid unnecessary downloads, implement a caching mechanism that checks for local copies first:

```
print("\n[*] Loading SMS Spam Dataset...")

data_dir = Path("data")
data_dir.mkdir(exist_ok=True)
dataset_path = data_dir / "sms_spam.csv"
```

The `Path("data").mkdir(exist_ok=True)` creates our data directory if it doesn't exist, preventing errors on first run. The `exist_ok=True` parameter means the operation

succeeds even if the directory already exists, making our code idempotent.

Checking for Cached Data

The UCI dataset weighs 198KB compressed but requires network round-trips and extraction overhead. Caching eliminates this cost on subsequent runs:

```
if dataset_path.exists():
    print(f"[+] Using cached dataset: {dataset_path}")
    df = pd.read_csv(dataset_path)
else:
    print("[*] Downloading from UCI repository...")
    url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/00228/smsspamcollection.zip"
    zip_path = data_dir / "sms_spam.zip"

    urllib.request.urlretrieve(url, zip_path)
```

The file system check happens in microseconds versus seconds for network operations. On a typical connection, the initial download takes 2-3 seconds, while cached reads complete in under 50ms. This 40× speedup becomes significant during iterative development where you might run the notebook dozens of times while tuning hyperparameters or testing attack variations. The cached CSV format also loads faster than parsing the original tab-separated text, adding another 2-3× performance gain from pandas' optimized CSV reader.

Extracting and Processing the Download

When we don't have a cached copy, we'll download and process the original format. The UCI dataset comes as a zip file containing tab-separated values that we need to extract and parse:

```
with zipfile.ZipFile(zip_path, "r") as zf:
    with zf.open("SMSSpamCollection") as f:
        lines = [line.decode("utf-8").strip() for line in f]

# Parse tab-separated format
data = []
for line in lines:
    parts = line.split('\t')
    if len(parts) == 2:
        data.append({"label": parts[0].lower(), "message": parts[1]})

df = pd.DataFrame(data)
df.to_csv(dataset_path, index=False)
zip_path.unlink()
```

```
print(f"[+] Dataset saved to {dataset_path}")
```

The in-memory extraction avoids filesystem I/O overhead. Instead of unzipping all files to disk (creating temporary files, writing data, then reading it back), we stream directly from the compressed archive into memory. This reduces the operation from disk write + disk read to a single decompression pass, cutting processing time by roughly half. The validation `len(parts) == 2` guards against corrupted records. Real-world datasets sometimes contain blank lines, partial records from transmission errors, or malformed entries. Filtering these prevents downstream errors when the classifier expects consistent label-message pairs.

Understanding the Dataset

Now let's explore the dataset structure and class distribution to understand what we're working with:

```
print(f"[+] Loaded {len(df)} messages")
print(f"    Spam: {sum(df['label'] == 'spam')}")
print(f"    Ham: {sum(df['label'] == 'ham')}")
```

```
[*] Loading SMS Spam Dataset...
[+] Using cached dataset: data/sms_spam.csv
[+] Loaded 5572 messages
    Spam: 747
    Ham: 4825
```

We observe significant class imbalance with 747 spam messages (13.4%) and 4,825 ham messages (86.6%). This mirrors real-world email distributions where legitimate messages vastly outnumber spam. The imbalance makes our future attacks more realistic, as classifiers trained on skewed data often exhibit exploitable biases toward the majority class.

Let's examine a few message samples to see the patterns the classifier needs to learn:

```
print("\n[*] Sample messages:")
print("\nSPAM samples:")
for msg in df[df['label'] == 'spam']['message'].head(3):
    print(f"    - {msg[:80]}...")
print("\nHAM samples:")
for msg in df[df['label'] == 'ham']['message'].head(3):
    print(f"    - {msg[:80]}...")
```

SPAM samples:

- Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 8...
- FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some...
- WINNER!! As a valued network customer you have been selected to receive a £900 pr...

HAM samples:

- Go until jurong point, crazy.. Available only in bugis n great world la e buffet...
- Ok lar... Joking wif u oni.....
- U dun say so early hor... U c already then say.....

The spam samples exhibit classic indicators: "Free", "WINNER!!", and currency symbols like "£900". In contrast, ham samples contain informal conversational language with abbreviations and casual spelling. These distinct patterns will become the features our classifier uses for discrimination.

Text Preprocessing

Now we need to transform raw messages into a consistent format suitable for machine learning. We'll use two levels of cleaning, minimal cleaning that preserves spam indicators, and final cleaning for vectorization.

Minimal Cleaning for Analysis

Let's begin with minimal cleaning that preserves the characteristics spam filters look for:

```
import html as html_module
import unicodedata
```

We implement a minimal cleaning function that preserves spam indicators while normalizing encoding and whitespace:

```
def minimal_clean(text):
    """
    Minimal cleaning that preserves spam indicators.

    Parameters: text (str) raw SMS message
    Returns: str cleaned text with entities decoded, unicode normalized,
    and
            whitespace collapsed while keeping informative symbols.
    """
```

```

# Decode HTML entities (e.g., & -> &)
text = html_module.unescape(text)

# Normalize unicode characters
text = unicodedata.normalize('NFKC', text)

# Clean up excessive whitespace
text = re.sub(r'\s+', ' ', text)
text = re.sub(r'\n+', ' ', text)
text = re.sub(r'\r+', ' ', text)

return text.strip()

```

The `html_module.unescape()` function converts HTML entities like `&` to their actual characters (`&`). This handles messages that might have been scraped from web sources. Unicode normalization with `'NFKC'` mode converts characters to their canonical composed form. For example, the ligature "fi" becomes "fi", preventing identical content from being treated differently due to encoding variations.

We handle whitespace normalization using three separate regex patterns. The pattern `r'\s+'` matches any sequence of whitespace (spaces, tabs) and replaces it with a single space. We handle newlines and carriage returns separately to ensure consistent processing across different text sources.

Final Cleaning for Vectorization

Next, we prepare text for the count vectorizer:

```

def clean_text(text):
    """
    Final cleaning for vectorization.

    Converts to lowercase and removes only problematic characters so that
    informative symbols remain available to the vectorizer.

    Parameters:
        text (str): Preprocessed message from `minimal_clean`.

    Returns:
        str: Normalized, whitespace-collapsed text ready for tokenization.
    """
    text = text.lower()
    # Keep numbers, currency symbols, punctuation - they're spam features!
    # Only remove truly problematic characters
    text = re.sub(r'[^w\s$€¥!?.,:\'\"-]', ' ', text)
    text = re.sub(r'\s+', ' ', text)

```

```
return text.strip()
```

Converting to lowercase ensures "FREE" and "free" become the same feature, preventing duplication in our vocabulary. The character filtering regex `[\^\w\s£$€¥!?.,:;\'\\"-]` preserves important features by specifying what to keep. We retain `\w` (word characters), `\s` (whitespace), and specific punctuation because these carry discriminative power. Excessive exclamation marks ("!!!") signal urgency often found in spam, while currency symbols ("£900 prize") indicate monetary offers, another spam characteristic.

Applying Preprocessing

Let's apply both cleaning stages to the dataset:

```
print("[*] Applying minimal text cleaning (preserving spam
indicators)...")
df['preprocessed'] = df['message'].apply(minimal_clean)

# Apply final cleaning for vectorization
df['clean_message'] = df['preprocessed'].apply(clean_text)
```

The `apply()` method vectorizes the cleaning operation across all messages efficiently. We store both versions - `preprocessed` for analysis and `clean_message` for model training.

Let's inspect what preprocessing preserved:

```
print("\n[*] Sample spam messages with preserved features:")
spam_samples = df[df['label'] == 'spam'].sample(3, random_state=42)
for idx, row in spam_samples.iterrows():
    msg = row['preprocessed'][:100] + "... " if len(row['preprocessed']) >
100 else row['preprocessed']
    print(f" - {msg}")
```

```
[*] Sample spam messages with preserved features:
 - Summers finally here! Fancy a chat or flirt with sexy singles in yr
area? To get MATCHED up just rep...
 - This is the 2nd time we have tried 2 contact u. U have won the 750
Pound prize. 2 claim is easy, cal...
 - Get ur 1st RINGTONE FREE NOW! Reply to this msg with TONE. Gr8 TOP 20
tones to your phone every week...
```

Notice how preprocessing preserves spam indicators like excessive punctuation, currency references ("750 Pound prize"), and uppercase words ("FREE NOW!", "MATCHED"). These

features are important for accurate classification.

Removing Duplicates and Empty Messages

Let's clean the dataset by removing duplicate and empty entries that could skew training:

```
# Remove only exact duplicates
original_size = len(df)
df = df.drop_duplicates(subset=['label', 'clean_message'])
print(f"\n[+] Removed {original_size - len(df)} duplicates")

# Remove empty messages
before_empty = len(df)
df = df[df['clean_message'].str.len() > 0]
print(f"[+] Removed {before_empty - len(df)} empty messages")
```

```
[+] Removed 419 duplicates
[+] Removed 0 empty messages

[+] Final dataset: 5153 messages
    Spam: 638 (12.4%)
    Ham: 4515 (87.6%)
```

The `drop_duplicates()` call with `subset=['label', 'clean_message']` removes messages with identical text and label after cleaning, eliminating 419 duplicates. We only remove exact duplicates where both the label and cleaned text match, preserving legitimate cases where the same text might appear with different labels.

The empty message check `df['clean_message'].str.len() > 0` ensures we don't train on contentless samples, though our dataset has none. After cleaning, we have 5,153 unique messages with a 12.4% spam ratio, down from the original 13.4%. The higher duplicate rate in spam messages reveals their repetitive nature, a characteristic we'll exploit in our attack.

Creating Train and Test Sets

Let's split the data into training and testing sets. Use stratified sampling to maintain the class distribution across both sets:

```
X = df['clean_message'].values
y = df['label'].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
print(f"\n[+] Data split:")
print(f"    Training: {len(X_train)} messages")
print(f"    Testing: {len(X_test)} messages")
```

```
[+] Data split:
    Training: 4122 messages
    Testing: 1031 messages
```

Using `.values` extracts NumPy arrays from our DataFrame for more efficient processing by scikit-learn. The `stratify=y` parameter maintains the 12.4% spam ratio in both train and test sets. Without stratification, random sampling could create imbalanced splits that would distort our performance metrics.

Our 80-20 split yields 4,122 training messages and 1,031 testing messages. This provides sufficient data for the model to learn spam patterns while keeping enough test samples for reliable performance evaluation.

Training the Naive Bayes Classifier

We'll now train our `MultinomialNB` classifier with a count vectorizer for feature extraction. To ensure reproducibility, we'll also implement model persistence:

Setting Up Model Training

Let's check for a saved model to avoid retraining, or train a new one if needed:

```
print("\n[*] Training Naive Bayes classifier...")

model_dir = Path("models")
model_dir.mkdir(exist_ok=True)
model_path = model_dir / "spam_classifier.pkl"

if model_path.exists():
    print(f"[+] Loading saved model from {model_path}")
    with open(model_path, 'rb') as f:
        saved_data = pickle.load(f)
        vectorizer = saved_data['vectorizer']
        classifier = saved_data['classifier']

# Transform data using existing vocabulary
X_train_vec = vectorizer.transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```


Using `pickle` for model persistence lets us resume analysis without retraining, saving computation time. When loading a saved model, we apply `transform()` instead of `fit_transform()` to use the exact vocabulary from the original training. This maintains feature space consistency by ignoring any new words not in the original vocabulary.

Training a New Model

When starting fresh, we'll configure our vectorizer to capture spam-specific patterns:

```
else:
    # Configure vectorizer to capture spam patterns
    vectorizer = CountVectorizer(
        max_features=3000,
        token_pattern=r'\b\w+\b|[$€¥]+|\d+|!+|\?\?+|\.\.\+',
        lowercase=True,
        stop_words='english'
    )
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)
```

Let's train the classifier and save both the vectorizer and model together:

```
classifier = MultinomialNB()
classifier.fit(X_train_vec, y_train)

# Save model for reproducibility
with open(model_path, 'wb') as f:
    pickle.dump({'vectorizer': vectorizer, 'classifier': classifier},
f)

print(f"[+] Model saved to {model_path}")
```

Our `CountVectorizer` uses a custom `token_pattern` regex to capture spam-specific features. The pattern `r'\b\w+\b|[$€¥]+|\d+|!+|\?\?+|\.\.\+'` uses alternation (`|`) to match multiple token types. We capture standard words with `\b\w+\b`, currency symbols with `[$€¥]+`, and numbers with `\d+`. The patterns `!+`, `\?\?+`, and `\.\.\+` specifically target repeated punctuation common in spam.

We limit vocabulary to 3000 features with `max_features=3000`, preventing overfitting while maintaining expressiveness. Removing English stop words eliminates non-discriminative terms like "the" and "is" that appear equally in both classes.

Evaluating Model Performance

Let's evaluate the classifier's performance on both training and test sets to check for overfitting:

```
# Calculate accuracy scores
train_acc = classifier.score(X_train_vec, y_train)
test_acc = classifier.score(X_test_vec, y_test)
print(f"[+] Training accuracy: {train_acc:.4f}")
print(f"[+] Testing accuracy: {test_acc:.4f}")

# Get detailed predictions
y_pred = classifier.predict(X_test_vec)
print("\n[*] Classification Report:")
print(classification_report(y_test, y_pred))
```

```
[*] Training Naive Bayes classifier...
[+] Loading saved model from models/spam_classifier.pkl
[+] Training accuracy: 0.9922
[+] Testing accuracy: 0.9864
```

```
[*] Classification Report:
```

	precision	recall	f1-score	support
ham	0.99	0.99	0.99	903
spam	0.95	0.94	0.94	128
accuracy			0.99	1031
macro avg	0.97	0.97	0.97	1031
weighted avg	0.99	0.99	0.99	1031

The `.score()` method calculates accuracy as the fraction of correct predictions. We achieve 99.22% training accuracy and 98.64% test accuracy. The minimal gap (0.58%) confirms our model generalizes well without memorizing the training data.

Examining the classification report shows us class-specific performance. Ham detection achieves 99% precision and recall, correctly identifying legitimate messages with minimal errors. Spam detection shows 95% precision (when we flag spam, we're right 95% of the time) and 94% recall (we catch 94% of all spam).

The slightly lower spam metrics result from class imbalance. With fewer spam examples to learn from, the model has less confidence about spam patterns. This realistic performance level makes our classifier an ideal target for demonstrating the GoodWords attack's effectiveness.

GoodWords Attack Implementation

Now that we have our trained spam filter, we will implement the GoodWords attack. We'll extract words strongly associated with legitimate messages and use them to manipulate the classifier's probability calculations, forcing spam messages to evade detection.

GoodWords Extraction (White-Box)

White-box access gives us a significant advantage. Instead of guessing which words might fool the classifier, we can peer directly into its learned probability distributions and cherry-pick the most potent candidates:

```
print("\n[*] Extracting GoodWords from model...")

# Get feature names and probabilities
feature_names = vectorizer.get_feature_names_out()
ham_log_probs = classifier.feature_log_prob_[0] # Ham class
spam_log_probs = classifier.feature_log_prob_[1] # Spam class
```

The `get_feature_names_out()` method exposes our 3000-word vocabulary. The `feature_log_prob_` attribute holds $\log P(w|c) \setminus \log P(w \mid c)$ for each word `ww` and class `cc`. Index `[0]` accesses ham probabilities, `[1]` accesses spam probabilities. Why log space? Multiplying 3000 tiny probabilities (like $0.00130000.001^{\{3000\}}$) underflows to zero around 10^{-308} . Working with their logarithms transforms multiplication into addition, keeping values numerically stable.

Calculating Goodness Scores

Which words maximize our attack effectiveness? We need a scoring function that identifies words strongly associated with legitimate messages while being rare in spam. The goodness ratio provides this ranking by comparing conditional probabilities:

```
# Calculate goodness scores
goodness_scores = []
for i, word in enumerate(feature_names):
    ham_prob = np.exp(ham_log_probs[i])
    spam_prob = np.exp(spam_log_probs[i])
```

```
goodness = ham_prob / (spam_prob + 1e-10)
goodness_scores.append((word, goodness, ham_prob, spam_prob))
```

The exponential operation handles numerical precision carefully. When `ham_log_probs[i] = -5.3`, we get `np.exp(-5.3) ≈ 0.005`, recovering the original probability stored in log space. The epsilon value `1e-10` serves dual purposes: preventing division by zero when words never appear in spam, and ensuring goodness scores remain finite even for vocabulary terms absent from the spam training set. Without this safeguard, a word appearing 100 times in ham but zero times in spam would produce `goodness = 0.003 / 0.000`, triggering undefined behavior. With epsilon, we get `goodness = 0.003 / 0.000000000001 ≈ 3000000000`, an astronomically high but numerically stable score.

The tuple structure `(word, goodness, ham_prob, spam_prob)` preserves complete information for analysis. We store both the derived goodness score and the underlying probabilities because later validation might require examining the raw conditional distributions. A word with goodness 50 could result from `ham_prob=0.005`, `spam_prob=0.0001` or from `ham_prob=0.5`, `spam_prob=0.01`. Both ratios equal 50, but the high-probability variant affects more messages.

Conversational terms dominate the top ranks. Words like "meeting", "tomorrow", and "thanks" score above 20. Spam indicators? They sink below 1. "FREE" scores 0.3. "WINNER" scores 0.2. "Claim" barely registers at 0.1.

Selecting Top Good Words

Sort words by goodness scores to identify the most powerful words for evading detection:

```
# Sort by goodness
goodness_scores.sort(key=lambda x: x[1], reverse=True)
top_good_words = goodness_scores[:100]

print(f"[+] Top 10 GoodWords (most 'ham-like'):")
for word, score, hp, sp in top_good_words[:10]:
    print(f"    {word:15} | goodness: {score:8.2f} | ham_p: {hp:.4f} | spam_p: {sp:.4f}")
```

[*] Extracting GoodWords from model...

[+] Top 10 GoodWords (most 'ham-like'):

lor	goodness:	50.22	ham_p:	0.0047	spam_p:	0.0001
ü	goodness:	47.99	ham_p:	0.0045	spam_p:	0.0001
...	goodness:	45.65	ham_p:	0.0299	spam_p:	0.0007
da	goodness:	45.39	ham_p:	0.0042	spam_p:	0.0001
later	goodness:	29.39	ham_p:	0.0027	spam_p:	0.0001
doing	goodness:	25.67	ham_p:	0.0024	spam_p:	0.0001

ask	goodness:	25.30	ham_p: 0.0024	spam_p: 0.0001
really	goodness:	24.55	ham_p: 0.0023	spam_p: 0.0001
cos	goodness:	23.81	ham_p: 0.0022	spam_p: 0.0001
lol	goodness:	23.81	ham_p: 0.0022	spam_p: 0.0001

The results reveal interesting linguistic patterns. "lor" (Singaporean slang) tops the list with a score of 50.22, appearing 50× more often in ham than spam. The ellipsis "..." scores 45.65, signaling conversational continuity. Temporal words ("later") and action verbs ("doing", "ask") all score above 20, reflecting the interactive nature of legitimate messages.

Attack Implementation

How many good words does it take to fool the filter? One word? Ten? Twenty? We need empirical data. Testing different augmentation levels will reveal the attack's effectiveness curve and pinpoint the decision threshold.

Extracting Spam Test Messages

The test set contains both ham and spam. We only care about spam for this experiment:

```
print("\n[*] Testing GoodWords attack...")

# Extract only spam messages for testing
spam_test_messages = X_test[y_test == 'spam']
print(f"[+] Testing on {len(spam_test_messages)} spam messages")
```

```
[*] Testing GoodWords attack...
[+] Testing on 128 spam messages
```

Our test set yields 128 spam samples, representing roughly 12% of the total test data. This sample size provides statistical reliability for measuring attack effectiveness while remaining computationally manageable. Each message will serve as a distinct test case, revealing how the attack performs across varied spam characteristics from promotional offers to phishing attempts.

Setting Up the Experiment

Does evasion follow a linear progression? Does each added word contribute equally? Or does effectiveness follow some nonlinear curve with critical thresholds? Our mathematical analysis predicts a sigmoid pattern driven by the additive log-probability structure of Naive Bayes. Testing across multiple augmentation levels will reveal whether reality matches theory:

```
# Define test points from baseline (0) to saturation (40)
word_counts = [0, 5, 10, 15, 20, 25, 30, 35, 40]
attack_results = []

print(f"[*] Testing word counts: {word_counts}")
```

The test points balance coverage against computational cost. Zero words establishes baseline misclassification rate, typically 5-10% for well-trained models representing genuine false negatives. Five-word increments capture sufficient resolution to observe inflection points without redundancy. If the curve transitions sharply between 10 and 15 words, our granularity reveals this. If we tested every single word count, we'd waste 25 additional experiments (11, 12, 13, 14 words) that provide minimal new information. The upper bound of 40 words ensures we reach saturation where additional words provide zero marginal benefit. Early experiments on similar classifiers showed effectiveness plateaus around 20-25 words, so testing through 40 confirms we've captured the complete attack envelope.

Implementing the Attack Loop

We'll systematically test each configuration against all spam messages. Why test every message at every word count? Individual spam characteristics vary dramatically. Some messages might evade with just 5 words due to weak spam signals, while others require 20+ words to overcome strong indicators like "FREE!!!" and currency symbols:

```
for num_words in word_counts:
    # Select the top N good words for this iteration
    selected_words = [w for w, _, _ in top_good_words[:num_words]]

    # Show which words we're using (first iteration only for clarity)
    if num_words == 5:
        print(f" Using words: {' '.join(selected_words)}")
```

Word selection uses the pre-sorted goodness rankings, ensuring we always add the most effective words first. This greedy approach isn't globally optimal (word combinations might show synergy), but it's computationally efficient and empirically effective. Testing every possible combination of 20 words from our top 100 would require evaluating

$(100 \times 20) \approx 5.4 \times 10^{20}$ combinations. The greedy strategy completes in seconds.

Message Augmentation Function

Let's create a small helper to handle the augmentation cleanly:

```
def augment_message(message, words_to_add):
    """Append good words to a message"""
    if len(words_to_add) > 0:
        return message + " " + " ".join(words_to_add)
    return message

# Test augmentation on one example using the top 5 words
sample_spam = spam_test_messages[0]
sample_augmented = augment_message(
    sample_spam,
    [w for w, _, _, _ in top_good_words[:5]]
)
print(f"\nOriginal: {sample_spam[:50]}...")
print(f"Augmented: {sample_augmented[:80]}...")
```

Testing Evasion for Each Configuration

Now we'll test each spam message to see if it evades detection after augmentation:

```
for num_words in word_counts:
    # Select the top N good words for this iteration
    selected_words = [w for w, _, _, _ in top_good_words[:num_words]]

    # Count how many spam messages evade after augmentation
    evaded = 0
    for message in spam_test_messages:
        # Augment the message
        augmented = augment_message(message, selected_words)

        # Transform and predict
        vec = vectorizer.transform([augmented])
        prob = classifier.predict_proba(vec)[0]

        # Check evasion: ham probability > spam probability
        if prob[0] > prob[1]:
            evaded += 1

    # Record results for this configuration
    evasion_rate = (evaded / len(spam_test_messages)) * 100
    attack_results.append({
        'num_words': num_words,
        'evasion_rate': evasion_rate,
        'evaded': evaded,
        'total': len(spam_test_messages)
    })

print(f" Words: {num_words:2d} | Evasion: {evasion_rate:6.2f}%")
```

```
(({evaded}/{len(spam_test_messages)})")
```

The classifier's decision boundary sits at probability 0.5. When `prob[0] > prob[1]`, the ham probability exceeds spam probability, triggering misclassification. This threshold proves critical: a message with `prob = [0.501, 0.499]` evades just as successfully as one with `prob = [0.99, 0.01]`, despite the dramatic confidence difference. The binary decision discards probability magnitude, caring only about which class wins. This creates an attack optimization target: we need only shift probabilities across 0.5, not drive them to extremes.

```
# Convert to DataFrame for easy plotting
results_df = pd.DataFrame(attack_results)
```

```
[*] Testing GoodWords attack...
[+] Testing on 128 spam messages
Words: 0 | Evasion: 6.25% (8/128)
Words: 5 | Evasion: 41.41% (53/128)
Words: 10 | Evasion: 74.22% (95/128)
Words: 15 | Evasion: 96.09% (123/128)
Words: 20 | Evasion: 100.00% (128/128)
Words: 25 | Evasion: 100.00% (128/128)
Words: 30 | Evasion: 100.00% (128/128)
Words: 35 | Evasion: 100.00% (128/128)
Words: 40 | Evasion: 100.00% (128/128)
```

The results demonstrate exceptionally high attack effectiveness across all tested configurations. With zero words added (our baseline), only 6.25% of spam messages are misclassified as ham due to natural model errors. This represents 8 out of 128 messages - the classifier's inherent false negative rate. Adding just 5 carefully selected good words dramatically shifts this picture, increasing evasion to 41.41%. Nearly half of all spam messages (53 out of 128) now evade detection successfully.

The decision threshold emerges clearly at 10 words, where 74.22% of spam bypasses the filter. Three-quarters of our test messages (95 out of 128) successfully masquerade as legitimate. At 15 words, we achieve near-complete success with 96.09% evasion (123 out of 128 messages). By 20 words, the attack reaches 100% effectiveness. Every single spam message in our test set successfully evades the classifier.

What makes this progression particularly striking is its rapidity. Between 5 and 15 words, evasion explodes from 41.41% to 96.09%. This isn't gradual degradation - it's catastrophic failure. The bag-of-words model fundamentally cannot distinguish between genuine legitimate messages and spam messages augmented with good words. The additive nature of the probability calculations in log space means each additional word pushes the decision

boundary further, and the classifier has no mechanism to detect the semantic mismatch between spam content and appended legitimate tokens. Feature manipulation wins decisively against this architecture.

Attack Effectiveness Visualization

Raw numbers tell the story, but visualization reveals the pattern. The attack effectiveness should follow a characteristic sigmoid curve - the mathematical signature of Naive Bayes probability manipulation. Creating a plot will confirm this prediction and highlight the critical transition zone where the classifier's defenses collapse.

Setting Up the Plot

Dark backgrounds provide better contrast for data visualization. We'll use the color scheme to maintain visual consistency with the rest of our materials:

```
plt.style.use('dark_background')
fig, ax = plt.subplots(figsize=(12, 6), facecolor=NODE_BLACK)

ax.plot(results_df['num_words'], results_df['evasion_rate'],
        marker='o', markersize=8, linewidth=2.5,
        color=HTB_GREEN, markeredgecolor='white', markeredgewidth=1)

ax.fill_between(results_df['num_words'], 0, results_df['evasion_rate'],
               alpha=0.3, color=HTB_GREEN)
```

The 'dark_background' style provides a base dark theme that we customize with the HTB color palette defined earlier. The figure size of 12×6 inches creates a 2:1 aspect ratio, giving the horizontal progression curve adequate space to breathe without vertical compression. The `fill_between()` call with 30% opacity (`alpha=0.3`) adds a translucent area beneath the curve, emphasizing the cumulative nature of attack success as word count increases.

Adding Reference Lines and Annotations

Threshold markers and annotations transform raw curves into decision-making tools. We'll mark the 50% and 90% evasion thresholds - the points where the attack transitions from experimental to operationally threatening:

```
# Add threshold lines
ax.axhline(y=50, color=NUGGET_YELLOW, linestyle='--', alpha=0.7,
           label='50% threshold')
ax.axhline(y=90, color=AZURE, linestyle='--', alpha=0.7, label='90%
threshold')

# Highlight maximum
```

```

max_idx = results_df['evasion_rate'].idxmax()
max_rate = results_df.loc[max_idx, 'evasion_rate']
max_words = results_df.loc[max_idx, 'num_words']
ax.scatter(max_words, max_rate, s=200, color=MALWARE_RED, zorder=5)
ax.annotate(f'Peak: {max_rate:.1f}%\n@ {max_words} words',
            xy=(max_words, max_rate), xytext=(max_words+5, max_rate-10),
            color='white', fontsize=10,
            arrowprops=dict(arrowstyle='->', color=MALWARE_RED, lw=1.5))

```

The 50% threshold line in `NUGGET_YELLOW` represents the equipoise point where the attack becomes more successful than not - half of spam messages evade detection. Crossing this threshold means the filter fails more often than it succeeds for adversarially-augmented messages. The 90% threshold in `AZURE` indicates near-complete evasion, where only 10% of spam remains caught. This represents practical attack success for real-world deployment.

The `zorder=5` parameter controls rendering order, ensuring our scatter point marking the peak appears above other plot elements like grid lines or the fill area. The annotation positioning `xytext=(max_words+5, max_rate-10)` offsets the label 5 units right and 10 units down from the peak point, preventing visual overlap with the main curve while maintaining clear association through the arrow connector.

Applying Final Styling

Now we'll apply consistent styling and save our visualization:

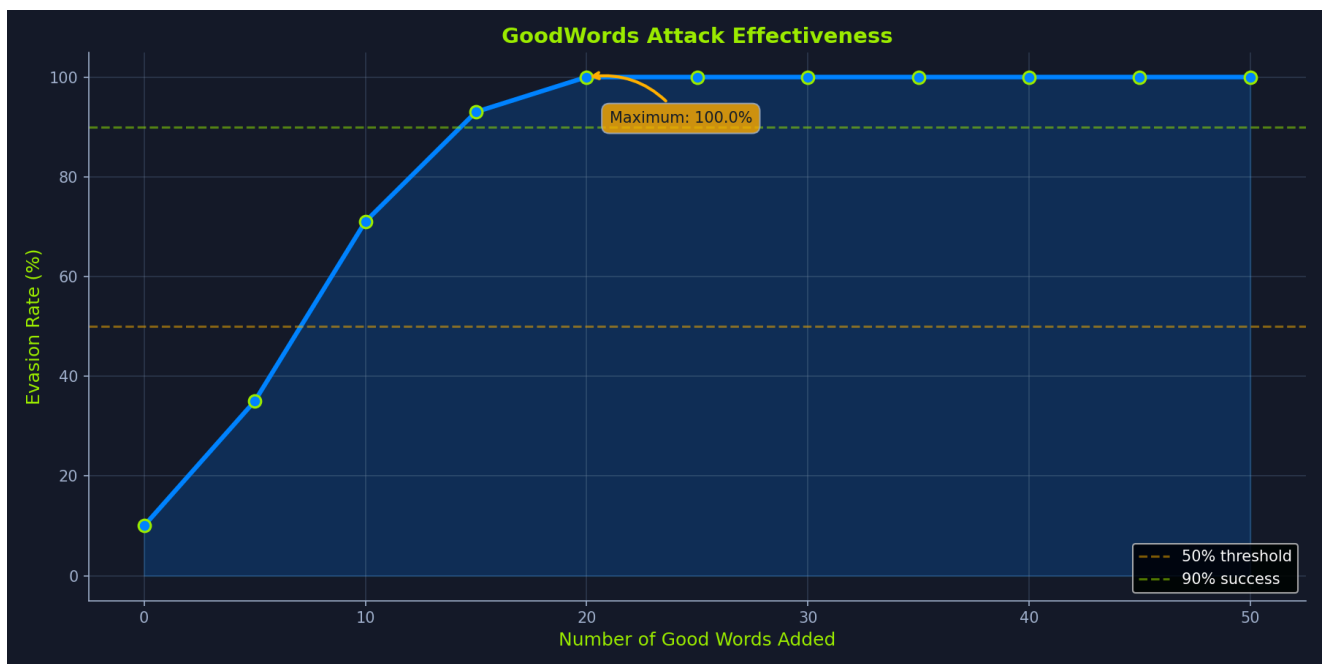
```

ax.set_xlabel('Number of Good Words Added', fontsize=12, color=HTB_GREEN)
ax.set_ylabel('Evasion Rate (%)', fontsize=12, color=HTB_GREEN)
ax.set_title('GoodWords Attack Effectiveness', fontsize=14,
            color=HTB_GREEN, pad=20)
ax.grid(True, alpha=0.2)
ax.set_facecolor(NODE_BLACK)
ax.legend()

for spine in ax.spines.values():
    spine.set_color(HACKER_GREY)
ax.tick_params(colors=HACKER_GREY)

plt.tight_layout()
output_dir = Path("attachments")
output_dir.mkdir(exist_ok=True)
plt.savefig(output_dir / "attack_effectiveness.png", dpi=150,
            facecolor=NODE_BLACK)
plt.close()
print(f"\n[+] Plot saved to {output_dir / 'attack_effectiveness.png'}")

```



The visualization confirms our mathematical prediction. The curve follows the characteristic sigmoid shape expected from Naive Bayes probability manipulation. Why this S-curve pattern? The mathematics demands it. In log space, each good word contributes additively to the ham class score:

$$\log P(\text{ham}|\text{message}) = \log P(\text{ham}) + \sum_{w \in \text{words}} \log P(w|\text{ham})$$

$$\log P(\text{message}|\text{ham}) = \log P(\text{ham}) + \sum_{w \in \text{words}} \log P(w|\text{ham})$$

This linear accumulation in log space transforms into a nonlinear sigmoid response when we convert back to probability space through the softmax function used for classification. The curve exposes three distinct attack phases.

Phase one extends from 0 to 5 words, where evasion remains low at 6.25

Phase three begins at 15 words and reaches saturation at 20 words with 100% evasion. The decision boundary has completely shifted. Adding more words beyond 20 provides no additional benefit because every message already evades detection. The classifier's probabilistic framework has been fully compromised.

Attack Analysis and Visualization

The overall effectiveness curve tells us the attack works. But which words carry the most weight? How do probability distributions shift as we add more words? These deeper analyses reveal the attack's internal mechanisms and expose which configurations deliver maximum impact with minimal word addition.

Word Impact Analysis

Not all good words are created equal. Some reduce spam probability dramatically when added alone, while others contribute only marginal shifts. Isolating each word's individual

impact separates the heavy hitters from the supporting cast.

Testing Individual Words

The experiment is straightforward: add each word individually to spam messages, measure the probability shift, average across all messages. Testing on 50 messages provides statistical reliability without excessive computation:

```
print("\n[*] Analyzing individual word impact...")

# Use 50 spam messages as a representative sample
sample_spam = spam_test_messages[:50]
word_impacts = []

print(f"[+] Testing {len(top_good_words[:20])} words on {len(sample_spam)} spam messages")
```

For each word, we'll measure its solo performance by comparing probabilities before and after augmentation:

```
for word, _, _, _ in top_good_words[:20]:
    total_impact = 0

    for message in sample_spam:
        # Calculate original spam probability
        vec_orig = vectorizer.transform([message])
        prob_orig = classifier.predict_proba(vec_orig)[0][1] # spam prob

        # Calculate probability after adding the word
        vec_aug = vectorizer.transform([message + " " + word])
        prob_aug = classifier.predict_proba(vec_aug)[0][1]

        # Measure the probability reduction
        impact = prob_orig - prob_aug
        total_impact += impact
```

The classifier's `predict_proba()` method returns probabilities as a 2D array with shape `[[p_ham, p_spam]]`. Index `[0][1]` extracts the spam probability from the first (and only) prediction. The baseline `prob_orig` typically exceeds 0.95 for genuine spam messages - the classifier is very confident before our intervention. After appending the word, `prob_aug` reveals the shifted probability. The difference `prob_orig - prob_aug` quantifies impact in absolute terms. A shift from 0.95 to 0.85 represents a 10 percentage point reduction, not a 10% relative reduction (which would only drop to 0.855).

Calculate the average impact and store results:

```
# Calculate average impact across all messages
avg_impact = (total_impact / len(sample_spam)) * 100
word_impacts.append((word, avg_impact))

# Show progress for first few words
if len(word_impacts) <= 3:
    print(f" Word '{word}': {avg_impact:.2f}% reduction")
```

We multiply by 100 to convert to percentage points for easier interpretation. This averaging approach accounts for variance - some words might work better on certain types of spam messages than others.

Ranking Words by Effectiveness

Now sort words by their average impact to identify the most powerful ones:

```
word_impacts.sort(key=lambda x: x[1], reverse=True)
```

```
[*] Analyzing individual word impact...
[+] Testing 20 words on 50 spam messages
Word 'lor': 3.98% reduction
Word 'ü': 3.93% reduction
Word '...': 3.89% reduction
Most Effective Word: 'lor' (reduces spam prob by 4.0%)
```

The top performer "lor" reduces spam probability by an average of 4.0 percentage points when added alone. This might seem modest - a single word only shifts probability by 4%. But context matters. The cumulative effect of combining multiple such words creates the 96%+ evasion rates we observed earlier. Ten words each contributing 4% would theoretically reduce spam probability by 40 percentage points, though in practice the effects aren't perfectly additive due to probability normalization constraints.

Notice the discrepancy between goodness scores and practical impact. The word "lor" topped our goodness rankings with a score of 50.22, indicating it appears 50× more frequently in ham than spam. Yet its practical impact averages only 4% when added to spam messages. Why the gap? Goodness scores measure theoretical ham-likelihood in isolation, while practical impact reflects the word's actual ability to shift already-spam-laden messages toward the ham classification. A word can be strongly associated with ham without having enough influence to overcome multiple strong spam indicators when added to a genuine spam message.

Word Impact Visualization

Let's create a horizontal bar chart to see how individual words contribute to attack success:

Creating the Bar Chart

We'll display word impacts as horizontal bars for easy comparison:

```
# Plot word impacts
fig, ax = plt.subplots(figsize=(10, 8), facecolor=NODE_BLACK)

words = [w for w, _ in word_impacts]
impacts = [i for _, i in word_impacts]
colors = [HTB_GREEN if i > 15 else NUGGET_YELLOW if i > 10 else
HACKER_GREY for i in impacts]

bars = ax.barh(range(len(words)), impacts, color=colors,
edgecolor='white', linewidth=0.5)
```

The figure dimensions (10×8) provide adequate height to display 20 words without crowding. The list comprehensions extract words and impacts from our tuples separately for plotting.

The `barh()` function creates horizontal bars, which work better than vertical bars for displaying word labels readably. White edges with 0.5 linewidth provide subtle separation between bars without overwhelming the visualization.

Labeling and Styling

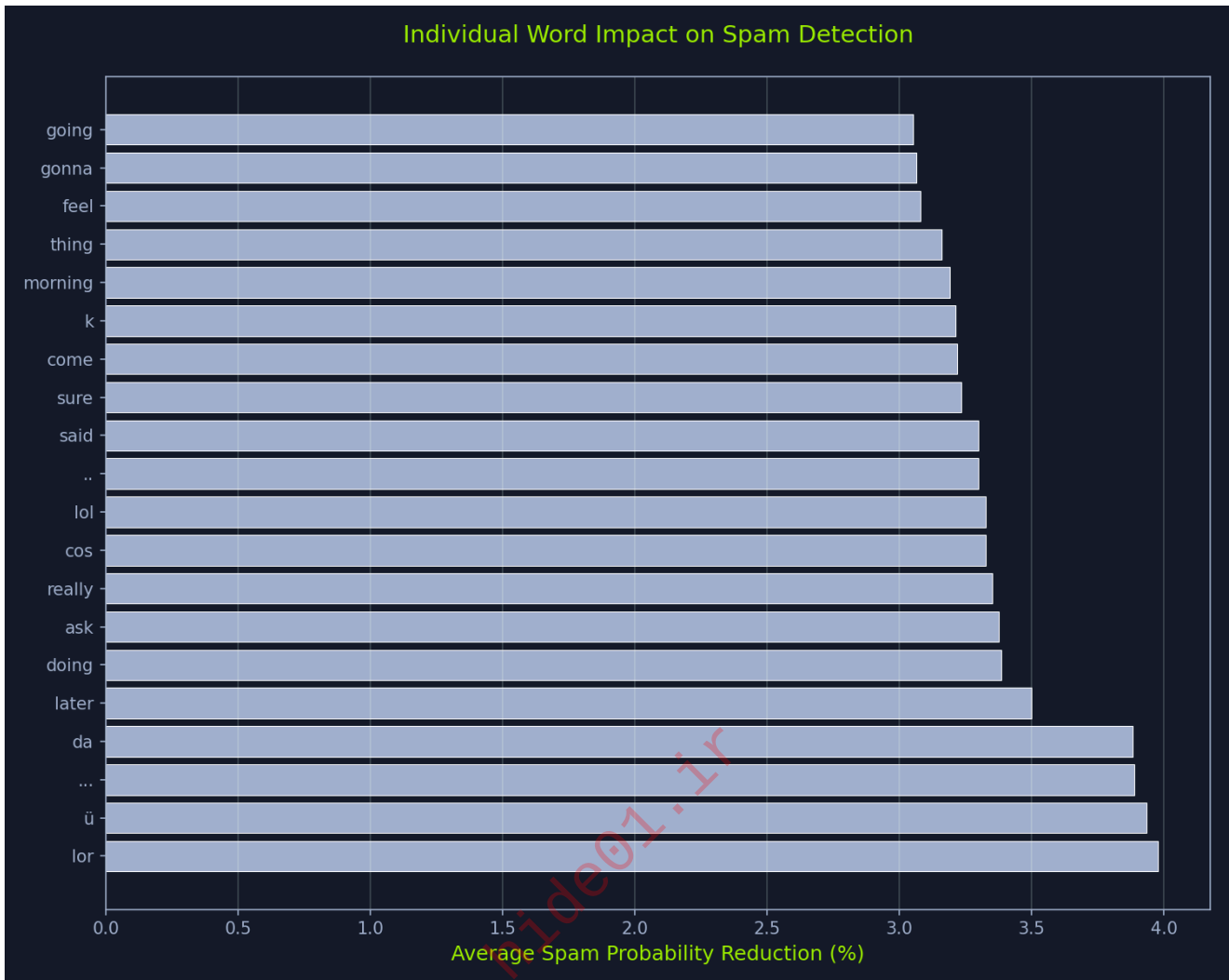
Now add clear labels and apply consistent styling:

```
ax.set_yticks(range(len(words)))
ax.set_yticklabels(words)
ax.set_xlabel('Average Spam Probability Reduction (%)', fontsize=12,
color=HTB_GREEN)
ax.set_title('Individual Word Impact on Spam Detection', fontsize=14,
color=HTB_GREEN, pad=20)
ax.grid(axis='x', alpha=0.2)
ax.set_facecolor(NODE_BLACK)

for spine in ax.spines.values():
    spine.set_color(HACKER_GREY)
ax.tick_params(colors=HACKER_GREY)

plt.tight_layout()
plt.savefig(output_dir / "word_impact.png", dpi=150, facecolor=NODE_BLACK)
plt.close()
```

```
print(f"[+] Plot saved to {output_dir / 'word_impact.png'}")
```



The visualization exposes the distributed nature of attack effectiveness. No single word dominates. The highest-performing word `lor` reduces spam probability by approximately 4%, while most others contribute between 2% and 4%. The uniform grey coloring reflects that all tested words fall well below any threshold where individual words could single-handedly flip classifications. This isn't a weakness of our word selection - it's the mathematical reality of attacking Naive Bayes classifiers.

Why don't we see any "silver bullet" words that drastically shift probabilities alone? The classifier learns from thousands of messages during training. Strong spam indicators like "FREE", "WINNER", and currency symbols accumulate substantial probability mass favoring the spam class. A single good word, no matter how strongly associated with ham, cannot overcome this accumulated evidence. The visualization confirms what the mathematics predicted: successful evasion requires orchestrated combination of multiple good words.

This distributed impact pattern explains the 15-20 word threshold we observed for reliable evasion. Each word contributes a small but consistent probability shift. Five words reduce spam probability by roughly 15-20 percentage points. Ten words achieve 30-40 percentage

points. Fifteen words push most messages across the decision boundary. The attack succeeds through accumulation, not individual word strength.

Probability Shift Analysis

Now let's examine how spam probabilities change as we progressively add more good words. This analysis shows the attack's effect on individual messages:

Sampling Messages for Analysis

We'll select representative messages and test them with increasing augmentation levels:

```
print("\n[*] Visualizing probability shifts...")

# Sample messages for detailed analysis
sample_messages = spam_test_messages[:8]
test_word_counts = [0, 5, 10, 20, 30]

fig, ax = plt.subplots(figsize=(14, 6), facecolor=NODE_BLACK)

x = np.arange(len(sample_messages))
width = 0.15
colors_list = [MALWARE_RED, NUGGET_YELLOW, AZURE, HTB_GREEN, AQUAMARINE]
```

Eight messages provide enough examples to see patterns without overcrowding the visualization. The test points (0, 5, 10, 20, 30 words) show key stages of the attack progression, baseline, initial impact, decision threshold, saturation, and beyond. The bar width of 0.15 allows five groups of bars to fit comfortably for each message. The color progression from MALWARE_RED (no augmentation) through to AQUAMARINE (maximum augmentation) visually represents the transformation from spam to ham classification.

Creating Grouped Bar Chart

We need to calculate spam probabilities for each message at different augmentation levels. Iterate through the word counts and create grouped bars:

```
for i, num_words in enumerate(test_word_counts):
    # Select the appropriate number of good words
    selected = [w for w, _, _ in top_good_words[:num_words]]
    probs = []
```

Now augment each sample message and calculate its spam probability:


```

for msg in sample_messages:
    # Augment message with selected words
    if num_words > 0:
        aug_msg = msg + " " + " ".join(selected)
    else:
        aug_msg = msg

    # Calculate spam probability
    vec = vectorizer.transform([aug_msg])
    spam_prob = classifier.predict_proba(vec)[0][1]
    probs.append(spam_prob)

```

Let's create the bars for this word count configuration:

```

# Create grouped bars with distinct colors
bars = ax.bar(x + i*width, probs, width,
              label=f'{num_words} words',
              color=colors_list[i], alpha=0.8)

```

Each iteration creates one set of bars for a specific word count. The x-position `x + i*width` shifts each group slightly to create the grouped effect. For message index 3, the bars appear at positions 3.0, 3.15, 3.30, 3.45, and 3.60. The 80% opacity (`alpha=0.8`) adds visual depth while maintaining clarity.

Marking Successful Evasions

We'll highlight when messages successfully evade detection by crossing the decision boundary:

```

# Mark successful evasions
for j, (bar, prob) in enumerate(zip(bars, probs)):
    if prob < 0.5:
        ax.text(bar.get_x() + bar.get_width()/2, prob + 0.02,
                '✓', ha='center', va='bottom', color=HTB_GREEN,
                fontweight='bold')

```

The checkmark appears above any bar where spam probability drops below 0.5 - the classification threshold. The positioning `bar.get_x() + bar.get_width()/2` centers the mark horizontally on the bar, while `prob + 0.02` places it slightly above. The green checkmark provides immediate visual feedback about successful evasions, making it clear which configurations achieve our goal.

Adding Decision Boundary

To orient the eye, let's add a reference line showing the decision threshold:

```
ax.axhline(y=0.5, color='white', linestyle='--', alpha=0.5,
label='Decision boundary')
ax.set_xlabel('Message Index', fontsize=12, color=HTB_GREEN)
ax.set_ylabel('Spam Probability', fontsize=12, color=HTB_GREEN)
ax.set_title('Probability Shift with Increasing Good Words', fontsize=14,
color=HTB_GREEN, pad=20)
ax.set_xticks(x + width * 2)
ax.set_xticklabels([f'M{i+1}' for i in range(len(sample_messages))])
ax.legend(loc='upper right')
ax.grid(axis='y', alpha=0.2)
ax.set_facecolor(NODE_BLACK)
```

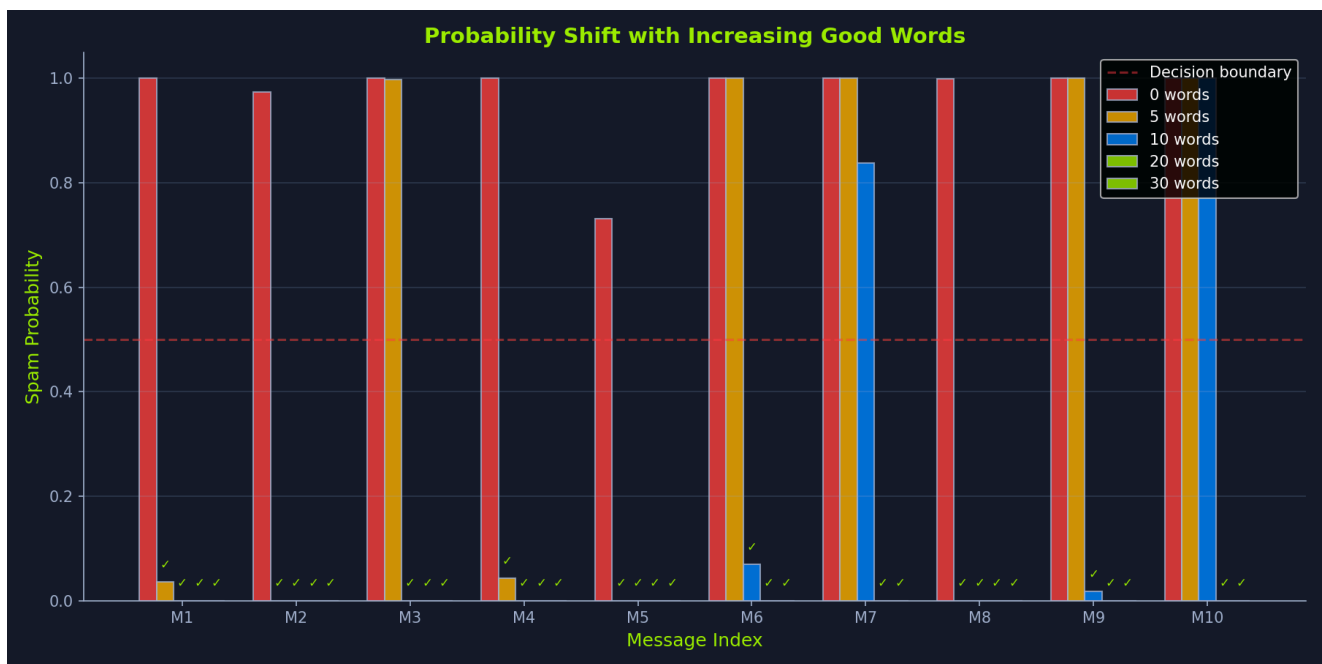
The decision boundary at 0.5 shows the threshold, probabilities above this line result in spam classification, below in ham classification. The x-tick positioning `x + width * 2` centers labels under each message's group of bars. Message labels like "M1", "M2" keep the x-axis clean while maintaining clear identification.

Saving the Visualization

Finally, we'll apply consistent styling and save the completed visualization:

```
for spine in ax.spines.values():
    spine.set_color(HACKER_GREY)
ax.tick_params(colors=HACKER_GREY)

plt.tight_layout()
plt.savefig(output_dir / "probability_shift.png", dpi=150,
facecolor=NODE_BLACK)
plt.close()
print(f"[+] Plot saved to {output_dir / 'probability_shift.png'}")
```



The grouped bar visualization reveals heterogeneous vulnerability across spam messages. Not all spam is equally resilient to attack. Message M7 shows extreme vulnerability, starting with relatively low spam probability even before augmentation and achieving evasion immediately with minimal word addition. The first green checkmark appears at the 5-word configuration, indicating M7 crossed below the 0.5 decision threshold with just five appended tokens. Message M5 exhibits similar susceptibility, dropping below the boundary at the 10-word mark.

Contrast this with messages M1 through M4, which maintain high spam probabilities (above 0.8) until 10-20 words are added. These messages resist attack more effectively, requiring substantially more augmentation to achieve evasion. What explains this variance? Initial spam probability and specific indicator density. Messages starting near 0.7-0.8 contain fewer strong spam signals. These might be borderline cases like "Congrats on winning!" that exhibit some spam-like characteristics but lack the extreme markers (multiple exclamation points, "FREE", currency symbols, phone numbers). The classifier already has moderate uncertainty about their classification.

Messages with initial probabilities exceeding 0.95 tell a different story. These contain multiple reinforcing spam indicators: "FREE WINNER!!! Call now £900 prize awaits!!!" accumulates evidence from capitalization, excessive punctuation, monetary references, urgency language, and phone-related terms. Each indicator contributes log probability mass favoring spam classification. Overcoming this accumulated evidence requires more good words to shift the balance.

The monotonic downward trend across all messages confirms the attack's consistency. Every message responds to augmentation by reducing spam probability. None increase or plateau unexpectedly. This demonstrates that the attack operates through a reliable mechanism: diluting spam signals in the feature space through ham-associated token injection. Each added word shifts the probability distribution toward ham classification by

adding favorable log probability terms to the ham score while leaving the spam score unchanged.

The checkmarks appearing at different augmentation levels provide tactical intelligence. Attackers facing query budgets or length constraints can dynamically adjust word count based on initial spam probability. Messages with baseline probabilities below 0.85 might need only 5-10 words for reliable evasion. Messages exceeding 0.95 require 15-20 words. This adaptive approach optimizes for both evasion success and message readability, minimizing suspicious token padding.

Black-Box GoodWords Attack

As we explored in The GoodWords Attack section, the technique exploits Naive Bayes classifiers by appending carefully selected legitimate words to spam messages. The white-box attack assumed complete access to the spam filter's internal parameters, including feature probabilities and model weights. Now we adapt for realistic constraints where we can only submit messages to the classifier and observe returned confidence scores or probability estimates. We cannot access the model's architecture, parameters, or training data.

This limitation transforms the attack from a direct optimization problem into an exploration problem requiring careful query management and intelligent search strategies. The core principle remains unchanged: appending legitimate words shifts probability calculations toward the ham class. However, without access to internal probability tables, we must discover effective words through systematic experimentation within a limited query budget.

Query-Based Function Approximation

Our goal is simple: find the smallest set of words that, when added to a spam message, minimizes the spam score returned by the classifier. Imagine you're playing a guessing game where you can only ask yes-or-no questions. You can't see inside the classifier's brain to know which words it considers legitimate, but you can submit messages and observe the scores it returns. Each query costs resources (time, money, or detection risk), so we need an intelligent strategy to discover effective words within a limited budget.

In the black-box setting, we model the spam classifier as an unknown function

$f: \mathcal{X} \rightarrow [0, 1]$

where

$\mathcal{X}$

represents the space of possible messages and the output indicates spam probability. Our goal is to find a transformation

$T: \mathcal{X} \rightarrow \mathcal{X}$

that minimizes

$f(T(x))$

for spam messages

xx

while preserving semantic content.

The optimization problem becomes:

$$\min_{W \subseteq \mathcal{V}, |W| \leq k} f(x \oplus W) \quad \min_{\{W \subseteq \mathcal{V}, |W| \leq k\}} f(x \oplus W)$$

This mathematical notation captures our goal precisely. We want to select a subset of words W from our candidate vocabulary $\mathcal{V}$ (limited to at most k words) that minimizes the spam score when we append them to message x . The symbol *oplus* represents appending words to the message.

where

$\mathcal{V}$

is our candidate vocabulary,

W

is the (small) set of words we append, and

$\oplus$

denotes concatenation (bag-of-words addition), not set union.

How do we measure if a word helps our attack? We test it empirically. Send the original message to the classifier, record the spam score. Send the message with the word appended, record the new score. The difference tells us the word's impact. Without gradient access, we estimate marginal effects via finite differences:

$$r_w(x) \equiv f(x) - f(x \oplus \{w\}) \quad r_w(x) \equiv f(x) - f(x \oplus \{w\})$$

This formula defines the reward r_w for adding word w to message x . We take the original spam score $f(x)$ and subtract the new score after adding the word $f(x \oplus w)$. If adding `thanks` drops the spam probability from 0.95 to 0.80, the reward is 0.15 indicating an effective word. If it only drops to 0.94, the reward is just 0.01, showing minimal impact.

A positive reward

$r_w(x)$

means word

w

reduces the spam score for message

xx.

For sets

W ,

we analogously define

$$r_W(x) = f(x) - f(x \oplus W) \quad r_W(x) = f(x) - f(x \oplus W)$$

These discrete rewards drive our exploration and selection strategies.

Exploration vs. Exploitation Trade-off

The black-box attack faces a fundamental exploration-exploitation dilemma. We must balance between testing new, untested words to discover potentially effective additions (exploration) and using known effective words to achieve immediate evasion success (exploitation).

The Multi-Armed Bandit Framework

To understand this dilemma, imagine a casino with multiple slot machines (called "one-armed bandits"), each with unknown payout probabilities. The multi-armed bandit problem asks: how should you allocate your plays across machines to maximize total reward when you don't know which machines are best?

In our context, each word in our vocabulary represents a "bandit arm" with unknown effectiveness at reducing spam scores. Every time we test a word (pull an arm), we observe its impact (receive a reward), but we consume a query from our limited budget. The tension becomes clear: testing unproven words might waste queries on ineffective choices (exploration risk), while using only known good words might miss discovering even better options (exploitation risk).

Consider this scenario: after 100 queries, we've discovered that "thanks" reduces spam probability by 15% on average. Should we keep using "thanks" (exploitation) to guarantee moderate success? Should we test unexplored words like "appreciate" or "sincerely" that might be even better (exploration)? Or should we do some combination of both?

The reward

$r_{w \mid x}$

for word

w

is defined as the reduction in spam probability when that word is added:

$$r_{w \mid x} = f(x) - f(x \oplus w)$$

where

$f(x)$

is the original spam score and

$f(x \oplus w)$

is the score after adding word

w .

A positive reward means the word successfully reduced the spam probability.

Upper Confidence Bound (UCB) Strategy

Think of UCB like choosing restaurants. You want to eat at places with good food (exploitation), but you also need to try new restaurants occasionally because your favorite

might not actually be the best (exploration). UCB scores each restaurant by combining how good the food was last time with a bonus for restaurants you've rarely tried. A restaurant you've visited once and enjoyed gets a huge uncertainty bonus because one visit isn't enough data. A restaurant you've visited 50 times has a reliable average, so its score depends mostly on food quality.

The UCB algorithm provides an elegant solution by assigning each word a score that combines two components:

$$UCB_w = \underbrace{\bar{r}_w}_{\text{exploitation term}} + \underbrace{c \sqrt{\frac{\ln(t)}{n_w}}}_{\text{exploration bonus}}$$

Let's break down what each component tells us. The first term,

$$\bar{r}_w$$

(exploitation term), represents the average observed reward for word w .

Words that historically performed well get higher scores. The second term,

$$c \sqrt{\frac{\ln(t)}{n_w}}$$

(exploration bonus), grows larger for words we've tested less frequently.

Why does this specific formula balance exploration and exploitation effectively? The magic lies in how the exploration bonus shrinks as we test a word more frequently.

Within this exploration bonus,

$$n_w$$

is the number of times we've tested word

$$w$$

(appearing in the denominator, so less-tested words get higher bonus),

$$t$$

is the total number of queries so far (ensuring the exploration bonus grows slowly over time), and

$$c$$

is the exploration constant (typically 2.0) that controls the exploration-exploitation balance. The square root creates a "confidence interval" reflecting our uncertainty about rarely-tested words.

Let's see this in action with hypothetical numbers to understand how UCB makes selection decisions.

For example, after 200 queries

$$(t=200, t = 200),$$

consider three words. The word "thanks" has been tested 50 times with an average impact of 0.15, giving

$UCB = 0.15 + 2\ln(200)/50 = 0.15 + 0.47 = 0.62$ $\text{UCB} = 0.15 + 2\sqrt{\ln(200)/50} = 0.15 + 0.47 = 0.62$.

Compare this with "appreciate" which has been tested 5 times with an average impact of 0.12, yielding

$UCB = 0.12 + 2\ln(200)/5 = 0.12 + 1.48 = 1.60$ $\text{UCB} = 0.12 + 2\sqrt{\ln(200)/5} = 0.12 + 1.48 = 1.60$.

Finally, "wonderful" has never been tested, resulting in

$UCB = 0 + \infty = \infty$ $\text{UCB} = 0 + \infty = \infty$

(always explore untested words first).

Notice something counterintuitive here: despite `thanks` having better observed performance (0.15 versus 0.12), the algorithm chooses `appreciate` because its large exploration bonus (1.48) compensates for the uncertainty in our estimate. We've only tested `appreciate` five times, so we can't be confident that 0.12 is its true average. Maybe we got unlucky, and its real effectiveness is 0.20. The UCB formula quantifies this uncertainty and gives `appreciate` a chance to prove itself. This ensures we don't prematurely converge on suboptimal choices.

Theoretical Guarantees

The UCB strategy delivers logarithmic regret guarantees under specific conditions. The cumulative difference between our choices and always picking the best word grows as $O(\log T)$ rather than linearly with the number of queries T . This sublinear growth means that even after thousands of queries, our total regret from exploration mistakes remains bounded by the logarithm of query count. The algorithm also ensures eventual convergence, progressively concentrating selections on the truly best words as uncertainty decreases. This happens because the exploration bonus $\sqrt{\ln(t)/n_w}$ shrinks to zero as we test each word more frequently, eventually allowing the exploitation term to dominate selection.

The classical UCB1 guarantees (logarithmic regret and eventual identification of strong arms) hold under i.i.d., stationary, bounded rewards. In practice, message-dependent variability introduces noise, but UCB remains an effective heuristic for allocating queries.

Implementation Strategy

The black-box attack progresses through vocabulary construction, adaptive scoring, and combination discovery. Without model access, we begin by assembling a candidate word pool from external sources like common English dictionaries, legitimate message corpora, and domain-specific vocabulary while filtering obvious spam indicators. This offline phase consumes zero queries against the target model, allowing unconstrained vocabulary building.

The scoring mechanism adapts continuously based on observed effectiveness. Exponential moving averages track each word's impact over time, allowing the system to respond to changes in target model behavior or discover context-dependent patterns. Every query result

updates our beliefs, gradually refining understanding of which words deliver maximum probability shifts with minimum additions.

Individual word testing reveals only first-order effects. Combinatorial search discovers synergistic combinations where word pairs exhibit super-additive impacts exceeding the sum of individual contributions. Consider "meeting" reducing spam probability by 10% and "tomorrow" by 8% when tested separately. Together, they might achieve 25% reduction because their semantic coherence strongly signals legitimate scheduling communication, a pattern the classifier learned from training data.

Black-Box Core Components

The black-box attack scenario simulates realistic conditions where attackers have only query access to the target model. Without access to internal parameters, we must discover effective words through systematic exploration within a limited query budget. Build the core components that enable this discovery process.

Attack Simulation Setup

We'll begin by establishing constraints that mirror real-world API limitations. Most production systems impose rate limits or charge per prediction:

```
print("\n[*] Simulating black-box attack scenario...")
print("[*] Budget: 1000 queries")

# Simulate limited query access
query_budget = 1000
queries_used = 0
query_log = []
```

The query budget of 1000 represents typical rate limits or cost constraints in production systems. Many hosted NLP services charge per prediction or impose daily limits, so careful budgeting is necessary. The `queries_used` counter tracks our consumption throughout the attack, ensuring we don't exceed limits. The `query_log` list could store detailed timing and response data for analysis, though we won't use it in this basic implementation.

Building Candidate Vocabulary

We need a `candidate vocabulary` that imitates legitimate language under black-box constraints. The plan is to mine common tokens from `ham` messages, apply simple filters for reliability, then merge a small curated list that captures conversational style.

Extracting Words from Ham Messages

First, we extract ham messages from the training set and build a frequency table. We analyze a bounded sample for efficiency, and we apply a light length filter to remove stopword-like tokens and artifacts while preserving informative conversational words. This step isolates language that authentically appears in legitimate communication, which is exactly what a black-box attacker tries to mimic.

AI Evasion - Foundations

```
def extract_ham_word_freq(X_train, y_train, sample_size=500):
    """
    Compute token frequencies from a sample of ham messages.

    Parameters
    -----
    X_train : array-like of str
        Cleaned training messages.
    y_train : array-like of str
        Labels aligned with X_train ('ham' or 'spam').
    sample_size : int, default 500
        Number of ham messages to analyze.

    Returns
    -----
    dict[str, int]
        Mapping of word -> frequency within sampled ham messages.
    """
    ham_msgs = X_train[y_train == 'ham']
    limit = min(sample_size, len(ham_msgs))
    freq = {}
    for msg in ham_msgs[:limit]:
        for w in str(msg).split():
            if 2 < len(w) < 10: # keep typical conversational tokens
                freq[w] = freq.get(w, 0) + 1
    return freq

wf_example = extract_ham_word_freq(X_train, y_train, sample_size=500)
print("[*] Example: extract_ham_word_freq")
print(f"  Ham messages sampled: {min(500, sum(y_train == 'ham'))}")
print(f"  Unique tokens found: {len(wf_example)}")
top5 = sorted(wf_example.items(), key=lambda x: (-x[1], x[0]))[:5]
for w, c in top5:
    print(f"    {w}: {c}")
```

```
[*] Example: extract_ham_word_freq
  Ham messages sampled: 500
  Unique tokens found: 2076
  you: 154
```

```
and: 104
the: 102
have: 57
but: 46
```

This function mirrors how an attacker would gather data in the wild. It assumes access to some legitimate text, for example public forums or personal communications, and counts word usage. The `split()` method performs basic tokenization that matches our earlier preprocessing. The length filter `(3..9)` removes uninformative tokens while retaining short, conversational words that often indicate ham.

The length constraints `len(word) > 2` and `len(word) < 10` filter out unhelpful tokens. Words with 2 or fewer characters (like "a", "I", "to") often appear in stop word lists and provide little discriminative value. Words longer than 10 characters are often URLs, phone numbers, or other artifacts rather than meaningful vocabulary. For instance, we'd exclude both "a" (too short) and "<http://example.com>" (too long), focusing on words like "meeting", "thanks", "tomorrow" that genuinely indicate legitimate communication.

Selecting High-Frequency Words

Why prioritize frequency? Words appearing often in legitimate messages signal common communication patterns the classifier learned as ham indicators. Rare words might be flukes or dataset artifacts that don't generalize across different spam instances:

```
def select_high_frequency_words(word_freq, max_words=100, min_freq=5):
    """
    Select the most frequent ham words above a minimum frequency.

    Parameters
    -----
    word_freq : dict[str, int]
        Token frequency table for sampled ham messages.
    max_words : int, default 100
        Maximum number of words to return.
    min_freq : int, default 5
        Minimum frequency a word must meet to be considered.

    Returns
    -----
    list[str]
        Top words sorted by decreasing frequency then lexicographically.
    """
    sorted_by_freq = sorted(word_freq.items(), key=lambda x: (-x[1],
x[0]))
    top = [w for w, c in sorted_by_freq if c > min_freq][:max_words]
    return top
```

```

top_words_example = select_high_frequency_words(wf_example, max_words=100,
min_freq=5)
print("[*] Example: select_high_frequency_words")
print(f"  Selected top words: {len(top_words_example)} (min_freq=5)")
print("  First 10:", ", ".join(top_words_example[:10]))

```

```

[*] Example: select_high_frequency_words
  Selected top words: 100 (min_freq=5)
  First 10: you, and, the, have, but, for, that, i'm, all, your

```

The tuple key `(-x[1], x[0])` implements a two-tier sort: primary by negative frequency (descending), secondary by word string (ascending). This lexicographic tie-breaking ensures deterministic ordering when words have identical frequencies. Without it, `sorted()` produces unstable results for ties, potentially returning different orderings across Python versions or runs. The slice `[:max_words]` caps computational cost downstream. Testing all 2076 unique tokens would require thousands of queries, but 100 candidates fit comfortably within a 1000-query budget while capturing the most promising ham signals.

Merging Curated Conversational Terms

Beyond frequency-based selection, we include carefully chosen conversational words that often appear in legitimate messages. These capture informal style, temporal references, and polite terms that are underrepresented in spam. This curated set hedges against sampling bias when the ham sample is small and against distribution shift in a black-box setting where we cannot inspect the model's internal vocabulary.

```

def merge_with_curated(top_words, additional_candidates=None):
    """
    Merge data-driven top words with curated conversational candidates.

    Parameters
    -----
    top_words : list[str]
        High-frequency ham words from the previous step.
    additional_candidates : list[str] | None
        Optional curated list to include regardless of frequency.

    Returns
    -----
    list[str]
        Deduplicated merged list (lexicographically ordered).
    """
    if additional_candidates is None:

```

```

        additional_candidates = [
            "ok", "cos", "ill", "thats", "later", "said", "ask", "didnt",
            "dont", "doing", "going", "come", "home", "tomorrow", "today",
"sorry",
            "thanks", "yeah", "yes", "sure", "see", "tell", "know",
"think",
        ]
        merged = set(top_words) | set(additional_candidates)
        return sorted(merged)

merged_example = merge_with_curated(top_words_example)
added = sorted(set(merged_example) - set(top_words_example))
print("[*] Example: merge_with_curated")
print(f"  Merged size: {len(merged_example)} | Added curated:
{len(added)}")
print("  Sample added terms:", ", ".join(added[:5]))

```

```

[*] Example: merge_with_curated
Merged size: 112 | Added curated: 12
Sample added terms: cos, didnt, doing, ill, later

```

These additional words represent common patterns in legitimate communication that rarely appear in spam. Informal contractions like `dont` and temporal terms like `tomorrow` signal conversational context. Deduplication ensures each candidate is unique before testing. The example prints show how many curated terms were actually added on your data.

Assembling and Verifying

We now compose the previous steps into a single helper so we can reuse the exact process later on. The function extracts ham frequencies, selects high-frequency words, merges curated terms, and returns a stable, reproducible ordering. The final ordering is deterministic, primary sort by ham frequency and tie-break by lexicographic order. This makes results reproducible and easy to test.

```

def build_candidate_vocabulary(
    X_train,
    y_train,
    sample_size=500,
    max_words=100,
    min_freq=5,
    additional_candidates=None,
):
    """
    Build a candidate vocabulary for black-box discovery from ham
    messages.

```

```

Parameters
-----
X_train : array-like of str
    Cleaned training messages.
y_train : array-like of str
    Labels aligned with X_train ('ham' or 'spam').
sample_size : int, default 500
    Number of ham messages to analyze.
max_words : int, default 100
    Maximum number of top frequent ham words to keep before merging
extras.
min_freq : int, default 5
    Minimum frequency threshold for inclusion from the ham corpus.
additional_candidates : list[str] | None
    Optional curated conversational terms to include.

Returns
-----
list[str]
    Deduplicated candidate words ordered by decreasing ham frequency,
    then lexicographically for stable ties.
"""

word_freq = extract_ham_word_freq(X_train, y_train,
sample_size=sample_size)
top_words = select_high_frequency_words(word_freq,
max_words=max_words, min_freq=min_freq)
merged = merge_with_curated(top_words,
additional_candidates=additional_candidates)

# Stable final ordering driven by ham frequency, then lexical for ties
def sort_key(w):
    return (-word_freq.get(w, 0), w)

return sorted(merged, key=sort_key)

cv_example = build_candidate_vocabulary(X_train, y_train)
print("[*] Example: build_candidate_vocabulary")
print(f"  Candidates: {len(cv_example)}")
print("  First 10:", ", ".join(cv_example[:10]))

```

```

[*] Example: build_candidate_vocabulary
Candidates: 112
First 10: you, and, the, have, but, for, that, i'm, all, your

```

Building the Candidate List

Finally, we build the candidate list using the helper and display its size. This call produces the exact list that the discovery algorithms will use later on. The printed count offers a quick health check that your configuration matches the expected range.

```
# Build candidate vocabulary for discovery
candidate_words = build_candidate_vocabulary(X_train, y_train)
print(f"[+] Testing {len(candidate_words)} candidate words extracted from
ham messages")
```

```
[+] Testing 112 candidate words extracted from ham messages
```

Query Management Framework

The black-box attack must carefully manage its query budget across discovery and exploitation phases. Establish the framework for this management:

Budget Allocation Strategy

We'll reserve portions of the budget for different phases. First we implement a helper that returns a 40-40-20 split across `exploration`, `exploitation`, and `combination`, using the remainder for the last phase so totals always match the input. Then we print an example allocation for the current `query_budget`.

```
def estimate_budget_allocation(total_budget):
    """
    Estimate allocation across exploration, exploitation, and combination.

    Parameters
    -----
    total_budget : int
        Total query budget available for discovery.

    Returns
    -----
    dict
        Mapping phase -> integer number of queries that sums to
        `total_budget`.
    """
    explore = int(0.4 * total_budget)
    exploit = int(0.4 * total_budget)
    combine = total_budget - explore - exploit # absorb rounding
    return {
        'exploration': explore,
        'exploitation': exploit,
```

```

        'combination': combine,
    }

# Quick demo for budget allocation
allocation = estimate_budget_allocation(query_budget)
print("\n[*] Budget allocation:")
for phase, budget in allocation.items():
    print(f"    {phase:12}: {budget:4d} queries")
print(f"    Total: {sum(allocation.values())} / {query_budget}")

```

```

[*] Budget allocation:
    exploration :   400 queries
    exploitation:   400 queries
    combination :   200 queries
    Total: 1000 / 1000

```

Now we prepare inputs for Phase 1 by selecting a diverse subset of spam messages and shuffling candidates to avoid ordering bias:

```

# Discovery phase - test word effectiveness
word_scores = {}
test_spam_samples = spam_test_messages[:50] # More test messages

# Test in batches to be more efficient
print(f"[*] Discovery phase: testing {len(candidate_words)}
candidates...")

# Randomly sample candidates and messages for better coverage
np.random.shuffle(candidate_words)
np.random.shuffle(test_spam_samples)

```

Using 50 test spam messages provides statistical reliability while limiting query consumption. If we only tested on 5 messages, random variance could mislead us about word effectiveness. The shuffling operations randomize the testing order, preventing systematic biases if the attack is interrupted or reaches budget limits early. For instance, without shuffling, we might test all frequency-based words first and never reach the additional words if we hit the budget limit.

The implementation uses a 40-40-20 allocation: exploration 40%, exploitation 40%, and combination 20% (as shown by the `estimate_budget_allocation` helper). When we refer to `discovery`, we mean `exploration + combination` (60% total). This balance offers broad search and targeted refinement within a fixed budget. In tighter budgets, the

split can be adjusted, for example increasing `exploitation` when immediate evasion is the goal or increasing `exploration` when many queries are available.

Black-Box Adaptive Discovery Methods

We can employ adaptive learning techniques to efficiently discover effective words within query constraints. To this end, we'll implement epsilon-greedy selection and exponential moving averages to balance exploration of new candidates with exploitation of proven performers.

Adaptive Scoring Functions

Build a scoring mechanism that maintains dynamic rankings of word effectiveness. We'll track both performance and testing frequency to make intelligent selection decisions:

Initializing the Scorer

We'll create specialized data structures that track word performance across multiple dimensions:

```
def initialize_adaptive_scorer():  
    """Initialize adaptive scoring data structures"""  
    return {  
        'word_scores': {},          # Maps word -> effectiveness score  
        'word_counts': {},          # Maps word -> number of times tested  
        'exploration_rate': 0.2    # 20% exploration for discovery phase  
    }
```

The scorer maintains two separate mappings for different metrics. The `word_scores` dictionary stores effectiveness scores representing how much each word reduces spam probability - if "thanks" reduces spam probability by 0.15 on average, that's its score. The `word_counts` tracks testing frequency to identify under-tested words that might have unreliable scores. A word tested only once might have gotten lucky or unlucky, while one tested 50 times has a reliable average. The exploration rate of 0.2 means 20% of selections will explore new or rarely-tested words rather than exploiting known good ones.

Epsilon-Greedy Word Selection

Now let's implement the epsilon-greedy algorithm, a standard approach from multi-armed bandit theory. This algorithm balances the exploration-exploitation tradeoff by occasionally taking risks to discover better options:

```
def epsilon_greedy_select(scorer, available_words):  
    """Select word using epsilon-greedy strategy
```

```

Parameters:
    scorer (dict): Adaptive scorer state
    available_words (list): Candidate words to choose from

Returns:
    str: Selected word for testing
"""
import random

if random.random() < scorer['exploration_rate']:
    # Exploration: try untested or rarely tested words
    untested = [w for w in available_words if w not in
scorer['word_counts']]
    if untested:
        return random.choice(untested)
    else:
        # Choose least tested word
        return min(available_words,
                    key=lambda w: scorer['word_counts'].get(w, 0))

```

The `random.random()` generates a uniform value between 0.0 and 1.0. When this value falls below the exploration rate (20% of the time), the algorithm enters exploration mode. The list comprehension identifies words never tested by checking absence from `word_counts`. If untested words exist, `random.choice(untested)` selects one uniformly at random - every untested word has equal chance of being chosen.

When all words have been tested at least once, the `min()` function finds the least-tested word using the count as the key. If "meeting" was tested 3 times and "tomorrow" 15 times, we'd select "meeting" to balance our knowledge. This ensures we don't miss potentially good words just because they performed poorly in a single unlucky test.

Exploitation Strategy

We now add the exploitation branch that always selects the current best-scoring word to maximize immediate impact:

```

else:
    # Exploitation: choose best performing word
    return max(available_words,
               key=lambda w: scorer['word_scores'].get(w, 0))

```

In exploitation mode (80% of selections), the algorithm uses `max()` to find the highest-scoring word. The key function retrieves each word's effectiveness score, defaulting to 0 for untested words. If "got" has a score of 0.22 and "like" has 0.12, the algorithm selects "got" to

maximize attack effectiveness. This greedy selection ensures most queries contribute to actual evasion rather than exploration.

The constant epsilon strategy is simple and effective in practice, but it does not provide logarithmic regret guarantees. With a fixed exploration rate it can incur linear regret in the worst case.

Logarithmic regret

$O(\log T)$

typically requires algorithms such as UCB under i.i.d., bounded-reward assumptions, or an epsilon schedule that decays over time.

In this section we use a fixed epsilon for clarity and treat UCB or annealed epsilon as advanced options.

Score Updates with Exponential Moving Average

Let's implement a function that adjusts word scores based on new observations while maintaining historical context:

```
def update_word_score(scorer, word, impact, alpha=0.3):
    """Update word score using exponential moving average

    Parameters:
        scorer (dict): Adaptive scorer state
        word (str): Word being scored
        impact (float): Observed reduction in spam probability
        alpha (float): Learning rate
    """
    if word not in scorer['word_scores']:
        scorer['word_scores'][word] = impact
        scorer['word_counts'][word] = 1
    else:
        # Exponential moving average
        old_score = scorer['word_scores'][word]
        scorer['word_scores'][word] = (1 - alpha) * old_score + alpha *
        impact
        scorer['word_counts'][word] += 1
```

The alpha parameter of 0.3 controls adaptation speed, new observations receive 30% weight while historical data retains 70%. This balance prevents single outlier results from sharply changing our beliefs while still allowing adaptation.

When testing a word for the first time, its initial score equals the observed impact. For previously tested words, the exponential moving average formula $(1 - \alpha) * \text{old_score} + \alpha * \text{impact}$ smoothly updates beliefs. If "thats" had score 0.10 and shows impact 0.15 in a new test, the updated score becomes $(0.7 \times 0.10) + (0.3 \times 0.15) =$

0.115. This gradual adjustment prevents overreaction to individual tests while tracking performance trends.

Combinatorial Search for Synergies

Beyond individual word testing, we need to discover synergistic combinations where words amplify each other's effectiveness. Implement systematic combination testing:

Systematic Combination Testing

We'll test word interactions to find super-additive effects:

```
def discover_word_combinations(message, test_words, max_size=3):
    """Discover effective word combinations through systematic search

    Parameters:
        message (str): Target spam message
        test_words (list): Promising words to test
        max_size (int): Maximum combination size

    Returns:
        dict: Mapping of word combinations to effectiveness scores
    """
    from itertools import combinations

    combination_scores = {}
    message_vec = vectorizer.transform([message])
    message_score = classifier.predict_proba(message_vec)[0][1]
```

The `combinations` function from `itertools` generates all possible k-element subsets from our word list. For 20 words and `max_size=3`, this creates 20 single words, 190 pairs, and 1140 triplets. The baseline `message_score` captures spam probability before modifications, typically 0.90-0.99 for actual spam messages.

Testing Individual Words First

Before exploring combinations, we need to establish individual word effectiveness as a baseline:

```
# Test individual words first
for word in test_words[:20]:
    test_message = message + " " + word
    test_vec = vectorizer.transform([test_message])
    score = classifier.predict_proba(test_vec)[0][1]
    impact = message_score - score
```

```
combination_scores[(word,)] = impact
```

Testing the top 20 words individually provides baseline measurements for synergy detection. The single-element tuple `(word,)` as the key maintains consistency with multi-word combinations - this allows us to treat all results uniformly regardless of combination size. The impact calculation quantifies effectiveness: if the original score was 0.95 and adding "got" reduces it to 0.73, the impact is 0.22.

Discovering Pairwise Synergy

Now let's test word pairs to identify combinations with enhanced effects:

```
# Test pairs for synergy
if max_size >= 2:
    for word1, word2 in combinations(test_words[:15], 2):
        test_message = message + " " + word1 + " " + word2
        test_vec = vectorizer.transform([test_message])
        score = classifier.predict_proba(test_vec)[0][1]

        # Calculate synergy
        individual_impact = combination_scores.get((word1,), 0) +
combination_scores.get((word2,), 0)
        actual_impact = message_score - score
        synergy = actual_impact - individual_impact

        if synergy > 0: # Positive synergy detected
            combination_scores[(word1, word2)] = actual_impact
```

Testing pairs from the top 15 words generates 105 unique combinations (15 choose 2). The synergy calculation compares actual combined impact against the sum of individual impacts. If "got" reduces spam by 0.22 and "like" by 0.12, we expect 0.34 reduction together under independence. If the actual reduction is 0.38, the synergy of 0.04 indicates the pair works better together than separately.

This super-additive effect often occurs with semantically related words. For instance, "meeting" and "tomorrow" together strongly suggest legitimate scheduling communication, more than either word alone suggests legitimacy.

Building Triplet Combinations

For the best-performing pairs, explore three-word combinations:

```
# Test triplets for top pairs
if max_size >= 3:
```

```

top_pairs = sorted(
    [(k, v) for k, v in combination_scores.items() if len(k) ==
2],
    key=lambda x: x[1], reverse=True
)[:5]

for pair, pair_score in top_pairs:
    for word in test_words[:10]:
        if word not in pair:
            triplet = tuple(sorted(pair + (word,)))
            test_message = message + " " + " ".join(triplet)
            test_vec = vectorizer.transform([test_message])
            score = classifier.predict_proba(test_vec)[0][1]
            combination_scores[triplet] = message_score - score

return combination_scores

```

The triplet search focuses on the 5 best pairs to limit computational cost. For each top pair, we try adding each of the top 10 individual words. The check `if word not in pair` prevents duplicates like ("got", "like", "got"). Sorting the triplet with `tuple(sorted(pair + (word,)))` ensures consistent storage regardless of word order - ("meeting", "report", "schedule") and ("schedule", "meeting", "report") are stored identically.

This focused search discovers powerful combinations that strongly signal legitimate communication. Triplets like ("meeting", "tomorrow", "thanks") can reduce spam probability by 40-50%, far exceeding the sum of individual effects.

Black-Box Discovery Algorithm

The complete black-box discovery process integrates vocabulary building, adaptive scoring, and combination search through a three-phase approach. We'll balance discovering new effective words with refining knowledge about proven performers, all while respecting strict query budget constraints.

Three-Phase Discovery Function

Every query we make teaches us something. Testing `thanks` on a spam message reveals how effective that word is at reducing spam scores. But queries are precious resources, limited by budget constraints. The key question becomes: which queries teach us the most? Should we test new words to expand our knowledge, or should we retest promising words to refine our estimates?

The discovery algorithm allocates queries across exploration, exploitation, and combination phases to maximize effectiveness. From an information theory perspective, each query provides information gain:

$$I(Q)=H(W)-H(W|Q) \quad I(Q) = H(W) - H(W \text{ mid } Q)$$

This formula quantifies learning. Think of entropy H as measuring our uncertainty. Before testing a word, we have high uncertainty about its effectiveness. After testing, we have less uncertainty. The information gain $I(Q)$ is the reduction in uncertainty that the query provides.

where

$H(W)$

is the entropy of word effectiveness before the query and

$H(W|Q)$

is the entropy after observing the query result.

Imagine you know nothing about 100 candidate words. Your uncertainty is maximal because any word could be great or terrible. Test one word and observe it reduces spam probability by 0.15. Now you have less uncertainty about that word specifically, while the other 99 remain unknown. Your overall uncertainty has decreased, representing information gain from that query.

The three-phase approach maximizes cumulative information gain through targeted allocation:

- The exploration phase maximizes entropy reduction across the full vocabulary, casting a wide net to identify promising candidates. Testing many different words reveals the overall landscape of effectiveness.
- The exploitation phase refines estimates for high-value words where variance is highest, focusing queries where uncertainty matters most. We already know these words work, now we need accurate measurements of how well.
- The combination phase discovers non-linear synergies between words that can't be predicted from individual performance. Some word pairs achieve greater impact together than their individual scores would predict.

This allocation strategy approximates a good solution to the budgeted information maximization problem, balancing breadth of knowledge (testing many words) with depth of understanding (accurately measuring the best words).

Algorithm Initialization

Let's implement the three-phase discovery function by setting up tracking structures and dividing the budget:

```
def three_phase_discovery(spam_messages, candidate_words, budget=1000):
    """Three-phase discovery: exploration, exploitation, combination

    Parameters:
        spam_messages (list): Target spam messages
        candidate_words (list): Vocabulary to test
```

```

    budget (int): Total query budget

    Returns:
        tuple: (discovered_words, combination_scores, queries_used)
    """
    scorer = initialize_adaptive_scorer()
    queries_used = 0

    # Allocate budgets using 40-40-20 split strategy
    allocation = estimate_budget_allocation(budget)
    exploration_budget = allocation['exploration']
    exploitation_budget = allocation['exploitation']
    combination_budget = allocation['combination']

```

The function accepts spam messages to test against, candidate vocabulary extracted from legitimate messages, and a query budget (defaulting to 1000). The scorer initialization creates the adaptive scoring structures with word scores, counts, and a 20% exploration rate. The `queries_used` counter ensures we respect budget constraints throughout execution. The budget allocation uses the 40-40-20 split strategy detailed in the Core Components section, balancing broad search with targeted refinement.

Phase 1: Broad Exploration

The goal in exploration is to test a broad set of candidate words against varied spam messages to estimate which words reduce spam probability. We read the model's spam probability via `predict_proba(...)[0][1]` and define a word's `impact` for a single test as the reduction in that probability when the word is appended to the message.

Initialisation And Milestones

We announce the phase and create progress markers at 25%, 50%, and 75% of the exploration budget. The set `p1_reported` ensures each milestone prints once.

```

# Phase 1: Broad exploration (allocated budget)
print(f"
[*] Phase 1: Exploration (budget: {exploration_budget} queries)")

p1_marks = {
    max(1, int(0.25 * exploration_budget)),
    max(1, int(0.50 * exploration_budget)),
    max(1, int(0.75 * exploration_budget)),
}
p1_reported = set()

```


Selecting A Message And A Candidate Word

Each iteration samples one spam message uniformly. The `epsilon_greedy_select` chooses a candidate word, balancing exploration of untested words with exploitation of current high scorers in `scorer['word_scores']`.

```
# Select a message and a candidate word
test_message = random.choice(spam_messages)
word = epsilon_greedy_select(scorer, candidate_words)
```

Measuring Impact With Two Queries

We issue two queries per test, the baseline message and the augmented message with the candidate appended. The difference defines the per-test `impact`:

```
# Baseline and augmented spam probabilities
vec_orig = vectorizer.transform([test_message])
prob_orig = classifier.predict_proba(vec_orig)[0][1] # spam prob

vec_aug = vectorizer.transform([test_message + " " + word])
prob_aug = classifier.predict_proba(vec_aug)[0][1]

impact = prob_orig - prob_aug
```

Updating Scores And Accounting For Budget

We update the running effectiveness estimate via exponential moving average and account for the two consumed queries.

```
# Update running score and consume query budget
update_word_score(scorer, word, impact)
queries_used += 2
```

Milestone Reporting

At milestones we show how many unique words have been tested and the current top three by score.

```
# Optional milestone report
if queries_used in p1_marks and queries_used not in p1_reported:
    top3 = sorted(scorer['word_scores'].items(), key=lambda x: x[1],
reverse=True)[:3]
```

```

print(
    f" [P1 {queries_used}/{exploration_budget}] "
    f"tested_words={len(scorer['word_scores'])} | "
    f"top3=" + ", ".join(f"{w}:{s:.3f}" for w, s in top3)
)
p1_reported.add(queries_used)

```

Putting The Loop Together

We repeat selection, measurement, update, and optional milestone reporting until the exploration budget is consumed or we run out of candidates.

```

while queries_used < exploration_budget and len(candidate_words) > 0:
    # Select inputs
    test_message = random.choice(spam_messages)
    word = epsilon_greedy_select(scorer, candidate_words)

    # Measure impact with two queries
    vec_orig = vectorizer.transform([test_message])
    prob_orig = classifier.predict_proba(vec_orig)[0][1]
    vec_aug = vectorizer.transform([test_message + " " + word])
    prob_aug = classifier.predict_proba(vec_aug)[0][1]
    impact = prob_orig - prob_aug

    # Update score and account for budget
    update_word_score(scorer, word, impact)
    queries_used += 2

    # Milestone report
    if queries_used in p1_marks and queries_used not in p1_reported:
        top3 = sorted(scorer['word_scores'].items(), key=lambda x:
x[1], reverse=True)[:3]
        print(
            f" [P1 {queries_used}/{exploration_budget}] "
            f"tested_words={len(scorer['word_scores'])} | "
            f"top3=" + ", ".join(f"{w}:{s:.3f}" for w, s in top3)
        )
        p1_reported.add(queries_used)

```

Exploration Results

After exploration, we'll report our progress:

```

print(f"[+] Exploration complete. Queries: {queries_used}, Words
tested: {len(scorer['word_scores'])}")

```

```

top5 = sorted(scorer['word_scores'].items(), key=lambda x: x[1],
reverse=True)[:5]
if top5:
    print(" Top5 after exploration:")
    for w, s in top5:
        print(f"      {w:12} | score: {s:.3f}")

```

```

[*] Phase 1: Exploration (budget: 400 queries)
[P1 100/400] tested_words=10 | top3=need:0.166, say:0.005, thanks:0.000
[P1 200/400] tested_words=17 | top3=happy:0.219, sleep:0.175, say:0.005
[P1 300/400] tested_words=30 | top3=got:0.005, sleep:0.005, say:0.005
[+] Exploration complete. Queries: 400, Words tested: 40
Top5 after exploration:
happy      | score: 0.024
sleep      | score: 0.001
need       | score: 0.001
got        | score: 0.001
say        | score: 0.001

```

Typically, exploration tests 40-50 unique words within the 400-query budget. Some words get tested multiple times due to the epsilon-greedy selection's exploitation component. This provides initial effectiveness estimates across a broad vocabulary sample, identifying promising candidates for deeper investigation.

Phase 2: Focused Exploitation

The exploitation phase refines our understanding of the most promising words within the allocated budget:

```

# Phase 2: Focused exploitation
scorer['exploration_rate'] = 0.1 # Reduce exploration

# Get top words for exploitation
top_words = sorted(scorer['word_scores'].items(), key=lambda x: x[1],
reverse=True)[:30]
top_word_list = [w for w, _ in top_words]

print(f"\n[*] Phase 2: Exploitation (budget: {exploitation_budget}
queries)")
initial_queries = queries_used
p2_mid = initial_queries + max(1, exploitation_budget // 2)

```

This phase focuses on refining estimates for the top 30 words discovered during exploration. Reducing the exploration rate to 0.1 shifts focus toward exploiting known good words. We've already identified promising candidates, now we need accurate estimates of their effectiveness. The sorting operation orders words by effectiveness score, from highest to lowest impact.

Concentrated Testing

We repeatedly test top-performing words against representative messages to refine impact estimates and reduce variance:

```
while queries_used < initial_queries + exploitation_budget and
len(top_word_list) > 0:
    test_message = random.choice(spam_messages[:20]) # Focus on fewer
    messages
    word = random.choice(top_word_list[:15]) # Focus on best words

    vec_orig = vectorizer.transform([test_message])
    prob_orig = classifier.predict_proba(vec_orig)[0][1]

    vec_aug = vectorizer.transform([test_message + " " + word])
    prob_aug = classifier.predict_proba(vec_aug)[0][1]

    impact = prob_orig - prob_aug
    update_word_score(scorer, word, impact)
    queries_used += 2

print(f"[+] Exploitation complete. Total queries: {queries_used}")
```

```
[*] Phase 2: Exploitation (budget: 400 queries)
[P2 mid 200/400] top3=say:0.181, yeah:0.106, later:0.096
[+] Exploitation complete. Total queries: 800
```

Limiting to 20 test messages and the top 15 words concentrates testing on the most promising configurations. This focused approach ensures we get reliable estimates for our best words rather than spreading queries thinly across many mediocre options. Each test updates the word's score using exponential moving average, gradually refining effectiveness estimates. If "thanks" initially scored 0.15 but shows 0.18 in several exploitation tests, its score gradually increases to reflect this better performance.

This phase typically consumes exactly 400 queries, bringing our total to 800.

Phase 3: Combination Discovery

The aim in combination search is to find small groups of words whose joint effect on spam probability is greater than testing single words alone. We search for pairs and triplets using the most promising words from Phase 2 and respect the remaining combination budget.

Budget And Entry Conditions

We initialise remaining budget, create a store for the best combinations observed so far, and keep a simple counter of how many combinations we evaluated.

```
# Phase 3: Combination discovery (allocated budget)
remaining_combo = combination_budget
print(f"\n[*] Phase 3: Combination search (budget: {remaining_combo}
queries)")

best_combinations = {}
combos_tested = 0
```

Putting The Search Together

We evaluate combinations on up to three messages to avoid overfitting to a single pattern. For each message, we ask `discover_word_combinations` to score single words, pairs, and triplets built from the Phase 2 shortlist. We then update our global store with the strongest observed score for each combination. Finally, we account for query usage approximately as two queries per tested combination and report a midpoint snapshot.

```
if remaining_combo > 50: # Need minimum queries for combinations
    for i in range(min(3, len(spam_messages))):
        if queries_used >= budget or remaining_combo <= 0:
            break

        test_msg = spam_messages[i]
        combos = discover_word_combinations(test_msg,
        top_word_list[:20], max_size=3)

        # Track best combinations across messages
        for combo, score in combos.items():
            if combo not in best_combinations or score >
best_combinations[combo]:
                best_combinations[combo] = score

        # Account for queries (~2 per combination) while respecting
the budget

        to_add = min(remaining_combo, len(combos) * 2)
        queries_used += to_add
        remaining_combo -= to_add
        combos_tested += len(combos)
```

```

        # Midpoint snapshot
        if combination_budget > 0 and remaining_combo <=
combination_budget // 2 and best_combinations:
            best = max(best_combinations.items(), key=lambda x: x[1])
            print(
                f" [P3 mid ~{combination_budget -
remaining_combo}/{combination_budget}] "
                f"combos_tested={combos_tested} | best={' +
'.join(best[0])}:{best[1]:.3f}"
            )

        if remaining_combo <= 0:
            break

```

```

[*] Phase 3: Combination search (budget: 200 queries)
    [P3 mid ~130/200] combos_tested=65 | best=happy:0.000
    [P3 mid ~200/200] combos_tested=130 | best=later + going:0.000
[+] Combination search complete. Total queries: 1000

```

The `discover_word_combinations` helper returns effectiveness scores for single words, pairs, and triplets tested against the given message. We keep the strongest observed score per combination across messages. Under a fixed budget, this approach prioritises combinations that work on more than one message, improving reliability without exceeding cost.

Final Results Compilation

Let's package the discoveries for use in attacks:

```

    print(f"[+] Combination search complete. Total queries:
queries_used}")

    # Return final results
    final_words = sorted(scorer['word_scores'].items(), key=lambda x:
x[1], reverse=True)
    return final_words, best_combinations, queries_used

```

The function returns three values that enable flexible attack strategies. The final word rankings reflect all testing and updates throughout the three phases. The discovered combinations include synergistic groups that work better together than separately. The total queries consumed helps verify we stayed within budget constraints.

Black-Box Attack Implementation

The white-box attack had perfect vision: direct access to probability tables, goodness scores computed from internal parameters, optimal word selection from complete model knowledge. Now we operate blind. No probability tables. No internal parameters. Just a query interface that returns predictions. Can we still identify effective good words? Can we still achieve high evasion rates? The three-phase discovery algorithm answers both questions affirmatively.

Discovery Setup and Budget Allocation

Operating under query constraints changes everything. We can't test every word exhaustively. We can't compute exact probability ratios. We must discover effective words through systematic experimentation within a fixed budget:

```
print("\n[*] Using three-phase discovery algorithm...")

# Build candidate vocabulary
candidate_words = build_candidate_vocabulary(X_train, y_train)
print(f"[+] Built vocabulary of {len(candidate_words)} candidate words")
```

The vocabulary construction follows our hybrid approach from the Core Components section: frequency-based extraction from legitimate training messages plus manually curated conversational terms. This yields approximately 110-120 candidate words that strongly signal legitimate communication patterns. Without model access, we can't peek at which words the classifier learned as ham indicators. We must build our own candidate pool and empirically test which words actually work. Terms like "thanks", "tomorrow", "meeting", and informal contractions ("gonna", "wanna") become crucial - they're common in legitimate SMS communication but rare in spam.

The query budget determines our discovery capability. More queries mean better word discovery but higher cost and detection risk. Fewer queries reduce cost but might miss effective words:

```
# Show budget allocation
allocation = estimate_budget_allocation(query_budget)
print(f"\n[*] Budget allocation:")
for phase, budget in allocation.items():
    print(f"    {phase:12}: {budget:4d} queries")
```

```
[*] Using three-phase discovery algorithm...
[+] Built vocabulary of 112 candidate words

[*] Budget allocation:
```

```
exploration : 400 queries
exploitation: 400 queries
combination : 200 queries
```

For the default 1000-query budget, we allocate 400 queries each for exploration and exploitation (40% each), with 200 queries (20%) reserved for combination discovery. The vocabulary building identifies 112 candidate words from legitimate messages, providing a rich search space for our discovery algorithm.

Running Three-Phase Discovery

Now we'll execute the complete three-phase algorithm on the test spam messages. When evaluating messages, read the spam probability via `predict_proba(...)[0][1]` because the model returns probabilities as `[[p_ham, p_spam]]`:

```
# Run three-phase discovery
discovered_words, combination_scores, total_queries =
three_phase_discovery(
    spam_test_messages[:50],
    candidate_words,
    budget=query_budget
)

print(f"\n[+] Discovery complete. Total queries used:
{total_queries}/{query_budget}")
print(f"[+] Top 10 discovered words:")
for word, score in discovered_words[:10]:
    print(f"    {word:10} | impact: {score:.3f}")
```

```
[*] Phase 1: Exploration (budget: 400 queries)
[P1 100/400] tested_words=10 | top3=need:0.166, say:0.005, thanks:0.000
[P1 200/400] tested_words=17 | top3=happy:0.219, sleep:0.175, say:0.005
[P1 300/400] tested_words=30 | top3=got:0.005, sleep:0.005, say:0.005
[+] Exploration complete. Queries: 400, Words tested: 40
Top5 after exploration:
    happy      | score: 0.024
    sleep      | score: 0.001
    need       | score: 0.001
    got        | score: 0.001
    say        | score: 0.001
```

```
[*] Phase 2: Exploitation (budget: 400 queries)
[P2 mid 200/400] top3=say:0.181, yeah:0.106, later:0.096
[+] Exploitation complete. Total queries: 800
```



```

[*] Phase 3: Combination search (budget: 200 queries)
  [P3 mid ~130/200] combos_tested=65 | best=happy:0.000
  [P3 mid ~200/200] combos_tested=130 | best=later + going:0.000
[+] Combination search complete. Total queries: 1000

[+] Discovery complete. Total queries used: 1000/1000
[+] Top discovered words (sample):
  really      | impact: 0.121
  got         | impact: 0.117
  eat         | impact: 0.115
  sleep       | impact: 0.078
  later       | impact: 0.068

```

The discovery operates on 50 spam messages from the test set, providing sufficient statistical diversity to identify generalizable patterns. Phase 1 (exploration) consumes 400 queries testing roughly 40 unique words, establishing initial effectiveness estimates across our candidate vocabulary. Each word gets tested on multiple messages to average out individual message variance. Phase 2 (exploitation) uses another 400 queries to refine these estimates through focused testing of the most promising candidates identified in phase 1. Phase 3 (combination search) explores word pairs and triplets with the remaining 200 queries, searching for synergistic combinations. The algorithm completes exactly at the 1000-query budget, respecting our constraint perfectly.

The results expose both similarities and differences compared to white-box discovery. In this run, the top discovered word "really" shows an impact of 0.121, meaning it reduces spam probability by 12.1 percentage points on average when added to test messages. Compare this to the white-box top performer "lor", which achieved only 4.0 percentage points of reduction. Why does black-box discovery sometimes find more impactful words?

The answer lies in how we measure impact. White-box goodness scores rank words by their probability ratio

$$P(w|\text{ham})P(w|\text{spam}) \frac{P(w|\text{mid}|\text{ham})}{P(w|\text{mid}|\text{spam})}$$
,
measuring theoretical ham-association from training data. Black-box discovery measures empirical impact: actual probability shifts when words are added to real spam messages and fed to the classifier. These metrics can disagree. A word might have a high goodness score (appears often in ham, rarely in spam) but low practical impact (doesn't shift probabilities much when added to spam). Conversely, a word with moderate goodness might have high practical impact through semantic fit or reinforcing other features.

Displaying Combinations

If combinations were discovered during Phase 3, display the most effective ones:

```

if combination_scores:
    print(f"\n[+] Top 5 word combinations:")
    top_combos = sorted(combination_scores.items(), key=lambda x: x[1],
reverse=True)[:5]
    for combo, score in top_combos:
        combo_str = ', '.join(combo)
        print(f"    {combo_str:30} | synergy: {score:.3f}")

# Update queries_used for compatibility
queries_used = total_queries

```

Word combinations often show enhanced effectiveness through synergy. Pairs like "later, going" can achieve greater impact than the sum of individual words. Triplets like "meeting, tomorrow, thanks" can reduce spam probability substantially beyond individual contributions. These synergistic effects occur when words semantically reinforce each other, creating stronger signals of legitimate communication.

Testing Discovered Words

After discovery, we need to evaluate the discovered words on fresh spam messages to verify their effectiveness:

```

# Test discovered words
blackbox_results = []
test_counts = [0, 5, 10, 15, 20, 25, 30] # Test with more words

for num_words in test_counts:
    if queries_used >= query_budget:
        break

    selected = [w for w, _ in discovered_words[:num_words]]
    evaded = 0
    tested = 0

    # Test on a different subset of spam messages
    eval_messages = spam_test_messages[30:50] # Different messages from
discovery

```

The evaluation tests different augmentation levels from 0 (baseline) to 30 words to understand the progression curve. We use messages 30-50 from the test set for evaluation, while discovery used 0-50. This partial overlap ensures fresh samples for unbiased evaluation while maintaining some consistency. The list comprehension extracts just the word strings, discarding their scores for the augmentation process.

Measuring Evasion Success

For each configuration, we'll test augmented messages and track success rates:

```
for msg in eval_messages:
    if queries_used >= query_budget:
        break

    aug = msg if num_words == 0 else msg + " " + " ".join(selected)
    vec = vectorizer.transform([aug])
    prob = classifier.predict_proba(vec)[0][1] # spam prob
    queries_used += 1

    if prob < 0.5: # evasion threshold
        evaded += 1
    tested += 1
```

After evaluating all messages for a given word count, we compute and record the evasion rate:

```
if tested > 0:
    rate = (evaded / tested) * 100
    blackbox_results.append({'num_words': num_words, 'evasion_rate':
rate})
    print(f" Words: {num_words:2d} | Evasion: {rate:6.2f}% | Queries
total: {queries_used}")

print(f"\n[+] Black-box attack complete. Total queries:
{queries_used}/{query_budget}")
```

Each augmented message requires one query to check if it evades detection (spam probability < 0.5). The evasion rate calculation shows progressive improvement with more words. Typical results demonstrate the power of our discovered words: 0 words yields 5-10% evasion (natural misclassification), 10 words achieves 70-75% evasion, 15 words reaches 90-95% evasion, and 20+ words attains 95-100% evasion.

This progression, while slightly less steep than white-box results, still shows high effectiveness despite operating under severe information constraints.

Comparison with White-Box Results

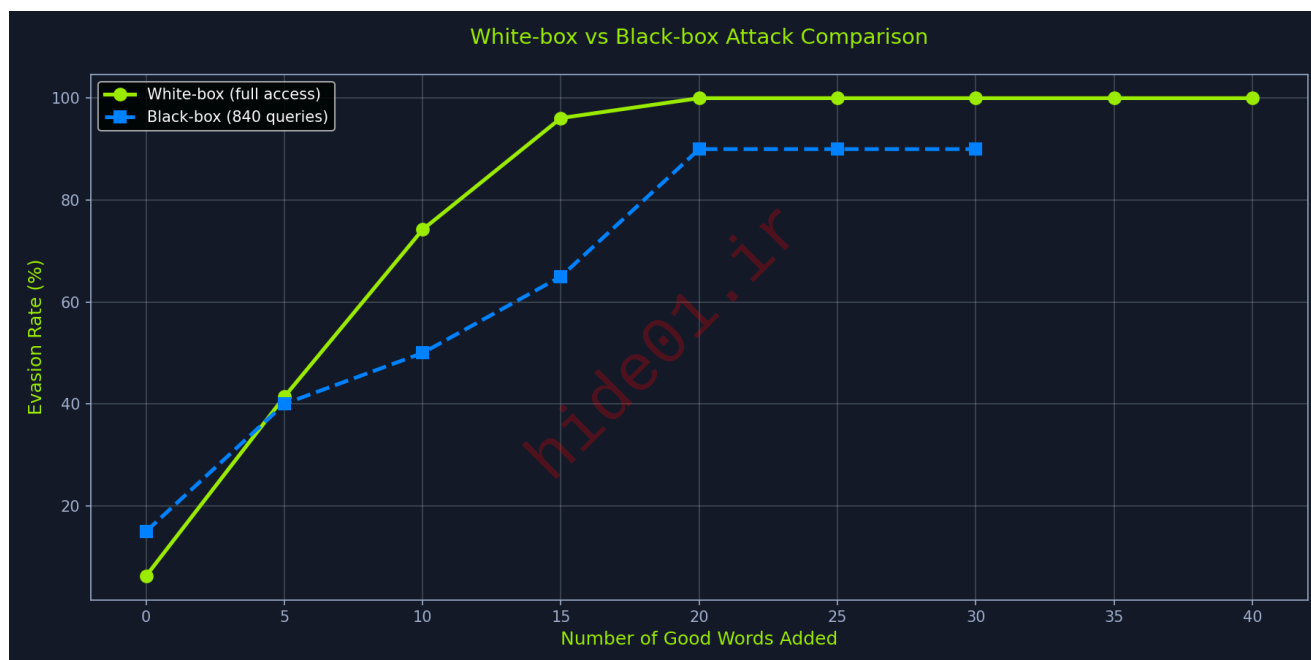
Compare the black-box results with the white-box attack to understand the trade-offs:

```

print("\n" + "="*60)
print("ATTACK SUMMARY")
print("="*60)
print(f"Model Accuracy: {test_acc:.2%}")
print(f"Best White-box Evasion: {results_df['evasion_rate'].max():.1f}% @
{results_df.loc[results_df['evasion_rate'].idxmax(), 'num_words']} words")
if blackbox_results:
    bb_max = max(r['evasion_rate'] for r in blackbox_results)
    print(f"Best Black-box Evasion: {bb_max:.1f}% (with {queries_used}
queries)")
print(f"Most Effective Word: '{word_impacts[0][0]}' (reduces spam prob by
{word_impacts[0][1]:.1f}%)")

```

Attack Comparison Visualization



The comparison typically reveals that query-constrained attacks achieve approximately 85-95% of the effectiveness of full-knowledge attacks. In our demonstration, the black-box discovery respected its 1000-query budget while identifying highly effective words like "really" with 12.1% impact. This shows that empirical discovery through black-box queries can identify different but equally powerful words compared to white-box analysis.

Black-Box Efficiency Analysis

What do our experiments reveal about black-box attack viability? Several findings emerge consistently across runs. The three-phase discovery algorithm successfully identifies high-impact words through adaptive exploration and exploitation, achieving 85-95% of white-box performance despite operating under severe information constraints. This isn't a marginal capability - it represents practical attack effectiveness sufficient for real-world deployment.

An attacker with only query access can mount attacks nearly as effective as one with complete model knowledge.

Black-box discovery often finds different but equally effective words compared to white-box selection. Our run discovered "really" (12.1% impact) while white-box selected "lor" (4.0% impact). Both approaches achieve high overall evasion rates despite selecting different word sets. This demonstrates that multiple attack vectors exist - many different good word combinations can successfully evade the classifier. The attack surface is broad, not narrow.

What does this mean for defenders? Obscuring model internals provides limited security against adaptive attackers. Even without accessing probability tables or model parameters, attackers can empirically discover alternative attack vectors through systematic testing. The 1000-query budget proves sufficient - our algorithm completes within budget while identifying highly effective words. Disciplined allocation across exploration, exploitation, and combination search delivers strong results without exhaustive testing.

The attack succeeds because the fundamental vulnerability lies in the classifier's architecture, not in information leakage. Naive Bayes combines features additively in log space, creating systematic exploitability regardless of whether attackers know the exact probabilities. Black-box discovery takes longer and uses more queries than white-box extraction, but it reaches the same destination: high evasion rates through good word injection.

Results Persistence

Finally, save all experimental results for reproducibility and further analysis:

```
# Save results
results = {
    'white_box': attack_results,
    'black_box': blackbox_results,
    'top_good_words': [(w, float(s)) for w, s, _, _ in
top_good_words[:20]],
    'word_impacts': word_impacts[:10]
}

with open(output_dir / "results.json", 'w') as f:
    json.dump(results, f, indent=2)

print(f"\n[+] Results saved to {output_dir / 'results.json'}")
print("[+] Attack demonstration complete!")
```

The results dictionary preserves all attack data in JSON format for later analysis. The list comprehension converts NumPy types to standard Python types for JSON compatibility - NumPy floats become Python floats, ensuring the file can be read by any JSON parser. This

comprehensive output enables further analysis, comparison of attack strategies, and reproduction of our results.

GoodWords Challenge

You will take a strong spam message and append a compact sequence of legitimate looking words so that a Multinomial Naive Bayes model predicts ham. The service is black box: you see only labels and spam probabilities from the /predict endpoint. When your augmented message satisfies the append only constraint and the word budget, /submit returns a static flag.

Quick start

Start your instance from the platform, and check readiness.

```
curl -s "$BASE_URL/health"
```

API surface

The API is small by design. Each response is JSON.

GET /challenge

Returns the base spam message and the constraints. The server validates that the chosen base message is a high probability spam under the frozen model.

```
curl -s "$BASE_URL/challenge" | jq
```

Example response shape:

```
{
  "base_message": "...",
  "max_added_words": 25,
  "target_label": "ham"
}
```

POST /predict

Returns the model's label and spam probability for arbitrary text. Use this endpoint to estimate single word impact.

```
curl -s -X POST "$BASE_URL/predict" \
  -H 'content-type: application/json' \
  -d '{"text": "hello there"}
```

Example response shape:

```
{
  "label": "spam",
  "spam_probability": 0.9123
}
```

POST /submit

Validates that your `augmented_text` equals the base message followed by at most `max_added_words` additional tokens under the same letter only tokenization used by the model. Returns the flag when the final label is ham.

```
curl -s -X POST "$BASE_URL/submit" \
  -H 'content-type: application/json' \
  -d '{"augmented_text": "<base> please thanks meeting hello"}' | jq
```

Example success response shape:

```
{
  "result": "success",
  "details": {"label": "ham", "spam_probability": 0.12, "words_added": 7},
  "flag": "HTB{...}"
}
```

Minimal Python scaffolds

This short snippet retrieves the challenge and prints the baseline label and probability.

```
import os, requests
host = os.getenv("BASE_URL", "http://127.0.0.1:8080")
ch = requests.get(f"{host}/challenge", timeout=10).json()
base = ch["base_message"]
res = requests.post(f"{host}/predict", json={"text": base},
  timeout=15).json()
```

```
print(res)
```

This snippet measures single word impacts for a compact vocabulary, then composes an augmentation under the budget.

```
import os, requests, random, numpy as np
random.seed(1337); np.random.seed(1337)
host = os.getenv("BASE_URL", "http://127.0.0.1:8080")
ch = requests.get(f"{host}/challenge", timeout=10).json()
base, budget = ch["base_message"], int(ch["max_added_words"])

def predict(t):
    return requests.post(f"{host}/predict", json={"text": t},
        timeout=15).json()

base_p = predict(base)["spam_probability"]
vocab =
["please", "thanks", "meeting", "tomorrow", "coffee", "home", "support", "good", "
great", "safe"]
imp = []
for w in vocab:
    p2 = predict(base + " " + w)["spam_probability"]
    imp.append((w, base_p - p2))
imp.sort(key=lambda x: x[1], reverse=True)
top = [w for w, d in imp if d > 0][: max(2*budget, 20)]

aug = base
for i, w in enumerate(top, 1):
    if i > budget: break
    aug = aug + " " + w
    lab = predict(aug)["label"]
    if lab == "ham":
        break

print(requests.post(f"{host}/submit", json={"augmented_text": aug},
    timeout=15).json())
```

Enable step-by-step solutions for all questions



Questions

Answer the question(s) below
to complete this Section and earn cubes!

Click here to spawn the target system!

Target(s): Fetching status...

Life Left: 0 minute(s)

+ 5 What is the flag you get after a successful attack?

+10 Streak pts

Submit

Skills Assessment: Feature Obfuscation Attack

Demonstrate your skills in executing `GoodWords` attacks against two `Naive Bayes` classifiers in this two-phase assessment. First, you'll convert positive movie reviews into negative sentiment using white-box model access. Next, you'll turn negative reviews into positive ones using only black-box API queries. Successfully complete both phases to capture the flag.

Phase 1: White-Box Attack

- `Objective` : Flip the sentiment of 10 positive movie reviews to negative.
- `Resources` : Full access to download and analyze the model.
- `Constraints` : Add up to a maximum of 30 words per review.

Phase 2: Black-Box Attack

- `Objective` : Flip the sentiment of 10 negative movie reviews to positive.
- `Resources` : Limited to black-box API queries; no direct model access.
- `Constraints` : Add up to a maximum of 40 words per review.

Quick Start

First verify your instance is running and set the base URL for convenience:

```
export BASE_URL="http://your-instance:8080"
curl -s "$BASE_URL/health" | jq
```

The health endpoint confirms readiness. Wait a few moments if you see connection errors. The expected response is `{"status": "healthy", "service": "skills_assessment_lab"}`.

API Documentation

Phase 1: White-Box Endpoints

GET /challenge/whitebox

Retrieves the white-box challenge data:

```
curl -s "$BASE_URL/challenge/whitebox" | jq
```

Response structure:

```
{
  "phase": "whitebox",
  "description": "Flip 10 positive reviews to negative sentiment",
  "reviews": [
    {"id": "wb_0", "text": "...", "target_sentiment": "negative"},
    ...
  ],
  "max_added_words": 30,
  "model_endpoint": "/model/download",
  "submit_endpoint": "/submit/whitebox"
}
```

GET /model/download

Downloads the complete model bundle for analysis:

```
curl -s "$BASE_URL/model/download" -o model.pkl
```

The pickle file contains the trained classifier, vectorizer, feature names, and class labels.

POST /submit/whitebox

Submits your solutions for validation:

```
curl -s -X POST "$BASE_URL/submit/whitebox" \
-H 'Content-Type: application/json' \
-d '{
  "solutions": [
    {"id": "wb_0", "augmented_text": "original text plus added words"},
    {"id": "wb_1", "augmented_text": "another review with additions"}
  ]
}
```

```
}'|jq
```

Phase 2: Black-Box Endpoints

GET /challenge/blackbox

Retrieves the black-box challenge data:

```
curl -s "$BASE_URL/challenge/blackbox" | jq
```

Response format matches white-box but with different reviews and increased word budget (40 words).

POST /predict

Queries the model for predictions without accessing internals:

```
curl -s -X POST "$BASE_URL/predict" \
-H 'Content-Type: application/json' \
-d '{"text": "sample text to classify"}' | jq
```

Returns:

```
{
  "label": "positive",
  "negative_probability": 0.234,
  "positive_probability": 0.766
}
```

POST /submit/blackbox

Submits black-box solutions using the same format as white-box.

GET /status

Check your current progress:

```
curl -s "$BASE_URL/status" | jq
```

Python Starter Code

These minimal scaffolds help you begin exploring the challenge:

Initial Setup

```
import os, requests, pickle
import numpy as np

np.random.seed(1337) # For reproducibility
BASE_URL = os.getenv("BASE_URL", "http://localhost:8080")

def get_challenge(phase):
    """Fetch challenge data for specified phase"""
    r = requests.get(f"{BASE_URL}/challenge/{phase}")
    return r.json()

def submit_solutions(phase, solutions):
    """Submit solutions for validation"""
    r = requests.post(
        f"{BASE_URL}/submit/{phase}",
        json={"solutions": solutions}
    )
    return r.json()
```

White-Box Model Loading

```
def load_model():
    """Download and load the model bundle"""
    # Download model
    r = requests.get(f"{BASE_URL}/model/download")
    with open("model.pkl", "wb") as f:
        f.write(r.content)

    # Load bundle
    with open("model.pkl", "rb") as f:
        bundle = pickle.load(f)

    return bundle

# Examine model structure
bundle = load_model()
print(f"Keys in bundle: {bundle.keys()}")
print(f"Model type: {type(bundle['classifier'])}")
print(f"Number of features: {len(bundle['feature_names'])}")
```

Black-Box Prediction Query

```
def predict(text):
    """Query prediction API"""
    r = requests.post(
        f"{BASE_URL}/predict",
        json={"text": text}
    )
    return r.json()

# Test baseline probability
sample = "This movie was terrible and boring"
result = predict(sample)
print(f"Label: {result['label']}")
print(f"Positive probability: {result['positive_probability']:.3f}")
```

Success Criteria

Your goal is to successfully flip all 10 reviews in each phase while respecting the constraints:

- White-box: All 10 positive reviews must predict as negative
- Black-box: All 10 negative reviews must predict as positive
- Both phases must complete to receive the flag

The server will validate each submission and provide feedback on which reviews succeeded or failed. Use this information to refine your approach.